TRAILHEAD
TECHNOLOGY PARTNERS

Oops, I Leaked It Again

API Security Mistakes Fixed

Jonathan “J.” Tower

```
{
   "celebrities":
      "Tom Hanks",
      "Beyoncé",
      "Leonardo DiCaprio",
      "Taylor Swift",
      "Dwayne Johnson",
      "Scarlett Johansson"
   }
```

Common API **security mistakes**, **practical fixes** to keep your REST APIs safe

TRAILHEAD
TECHNOLOGY PARTNERS

Jonathan "J." Tower

Partner & Principal Consultant

- Microsoft MVP in .NET
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- jtowermi
- Jonathan “J.” Tower

EXPERT
Consultation

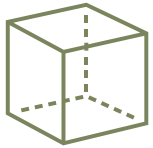
bit.ly/th-offer

github.com/trailheadtechnology/api-security

9 Most Common
API Security Mistakes

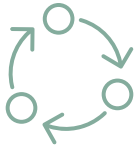
TRAILHEAD
TECHNOLOGY PARTNERS

Common Issues



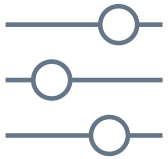
Broken Object Level Authz

Broken Object Prop Level Authz



Broken Function Level Authz

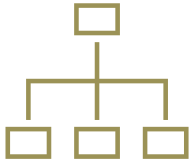
Broken Authentication



Misconfig & Insecure Defaults

Unrestricted Resource Consumption

Verbose Errors or Logging Leaks



Unrestricted Resource Consumption

Transport Layer & Secrets Mgmt

OWASP Top Ten

Main | Translation Efforts | Sponsors | Data 2025

Important note:

OWASP Top Ten 2025

Current project status as of Sept 2025:

- We are on track to announce the release of the **OWASP Top 10:2025** at the OWASP Global AppSec Conf in DC the first week of Nov 2025.

Stay Tuned!

The OWASP Top Ten Community Survey is open, please contribute your insights! Community Survey

The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications.

> Globally recognized by developers as the first step towards more secure coding.

The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Project Information

- OWASP Top 10:2021
- Making of OWASP Top 10
- OWASP Top 10:2021 - 20th Anniversary Presentation (PPTX)

Flagship Project
Documentation
Builder
Defender

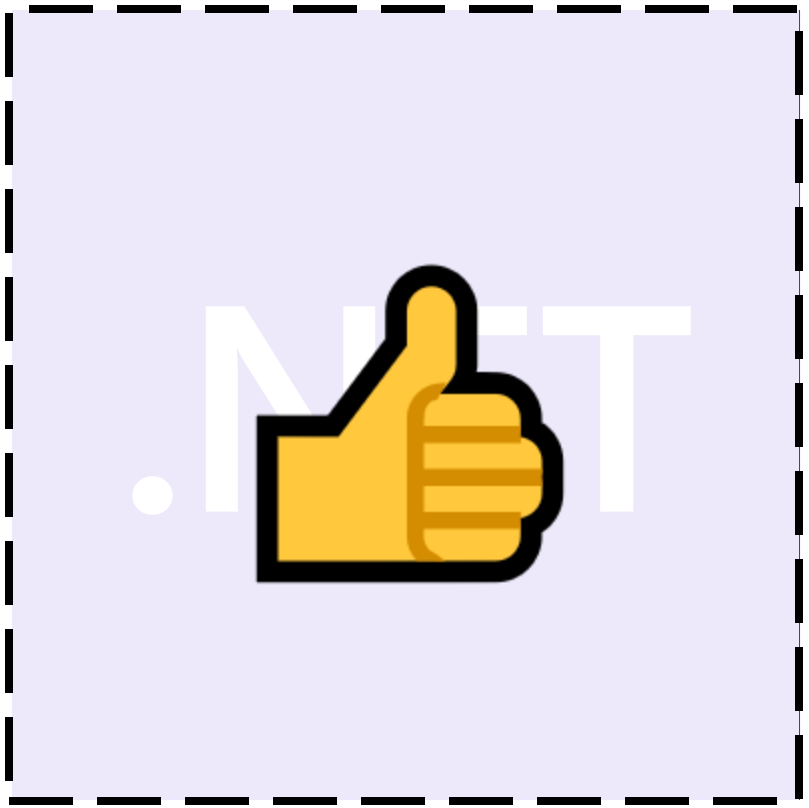
- Previous Version (2017)

Downloads or Social Links

- OWASP Top 10 2017
- Other languages → tab 'Translation Efforts'

Examples

Examples



Broken Object Level Authorization

Common Mistake #1

TRAILHEAD
TECHNOLOGY PARTNERS

Broken Object Property Level Authorization

Common Mistake #2

TRAILHEAD
TECHNOLOGY PARTNERS

Broken Object Property Level Authorization


```
{
  "id": 123,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",
  "ssn": "123-45-6789",
  "phoneNumber": "2125551212",
  "isAdmin": false,
  "lastLogin": "2025-09-25T13:45:00Z",
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."
}
```

Broken Object Property Level Authorization

Secrets
and
Credentials

```
{
  "id": 123,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",
  "ssn": "123-45-6789",
  "phoneNumber": "2125551212",
  "isAdmin": false,
  "lastLogin": "2025-09-25T13:45:00Z",
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."
}
```

Broken Object Property Level Authorization

Privilege Hints & Implementation Details

```
{
  "id": 123,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",
  "ssn": "123-45-6789",
  "phoneNumber": "2125551212",
  "isAdmin": false,
  "lastLogin": "2025-09-25T13:45:00Z",
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."
}
```

Broken Object Property Level Authorization

Heavy or
Unused Fields

```
{
  "id": 123,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",
  "ssn": "123-45-6789",
  "phoneNumber": "2125551212",
  "isAdmin": false,
  "lastLogin": "2025-09-25T13:45:00Z",
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."
}
```

Broken Object Property Level Authorization

Expose These Thoughtfully


```
{
  "id": 123,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",
  "ssn": "123-45-6789",
  "phoneNumber": "2125551212",
  "isAdmin": false,
  "lastLogin": "2025-09-25T13:45:00Z",
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."
}
```

Broken Object Property Level Authorization

Mass Assignment

```
public class User
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";


    public bool IsAdmin { get; set; }


    public Guid OwnerId { get; set; }
}
```

Solutions

Fixing Mass Assignment

Mass Assignment

```
public class User
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    [System.Text.Json.Serialization.JsonIgnore]
    public bool IsAdmin { get; set; }

    [Microsoft.AspNetCore.Mvc.ModelBinding.BindNever]
    public Guid OwnerId { get; set; }
}
```

Fixing Mass Assignment

Mass Assignment

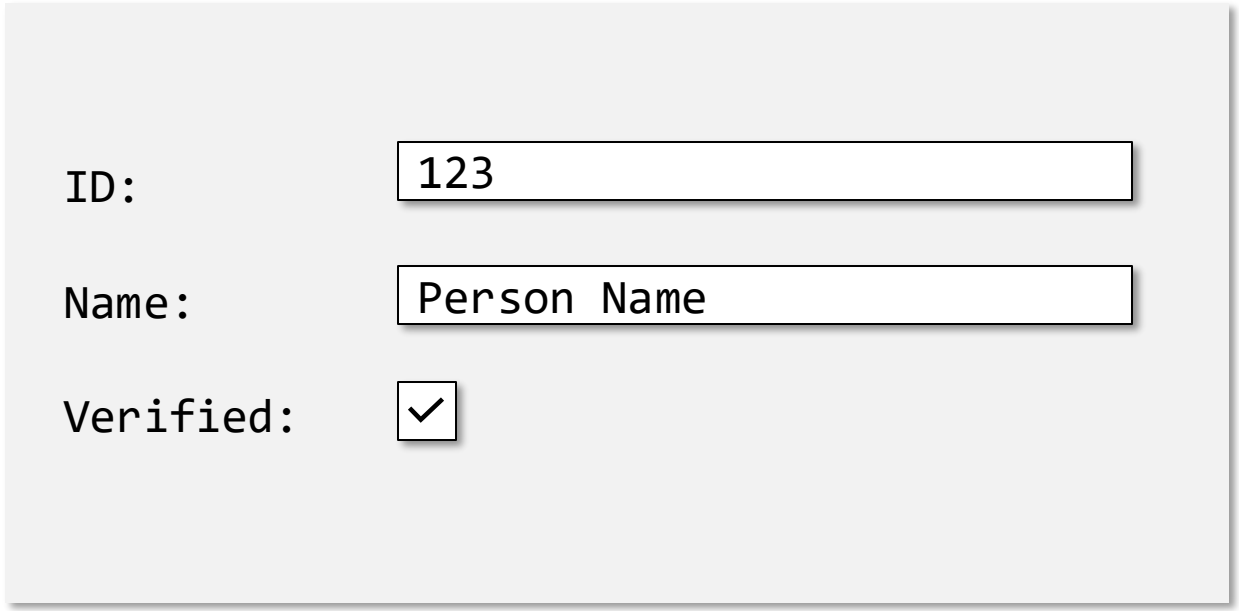
```
public class UpdateUserRequestDto
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";


    public bool IsAdmin { get; set; }


    public Guid OwnerId { get; set; }
}
```

Treat APIs Like UIs

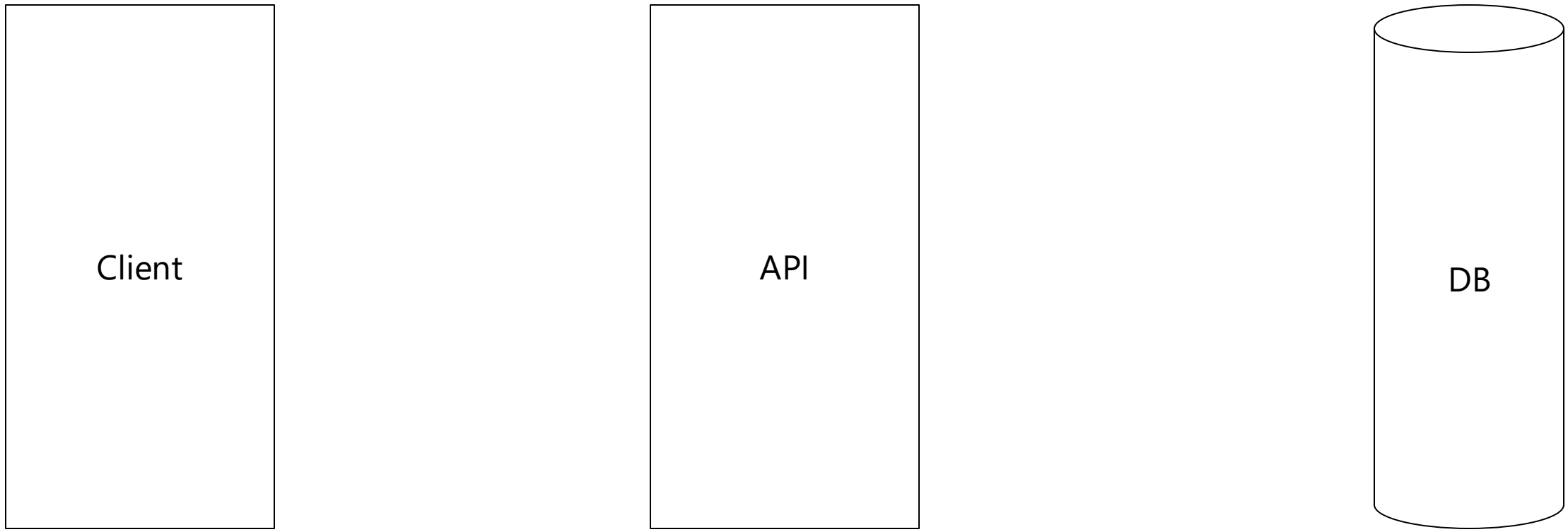
```
{
  "id": 123,
  "Name": "Person Name"
  "isDifficultUser": true
}
```

Treat APIs Like UIs

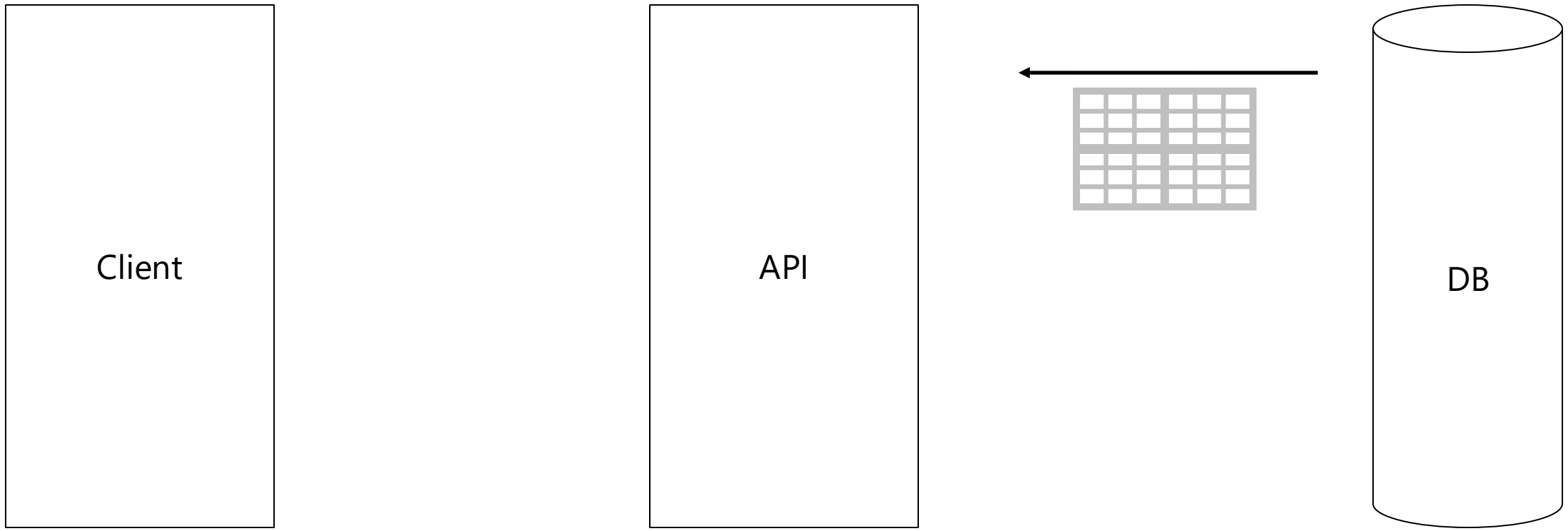
```
{
  "id": 123,
  "Name": "Person Name"
  "isVerified": true
}
```



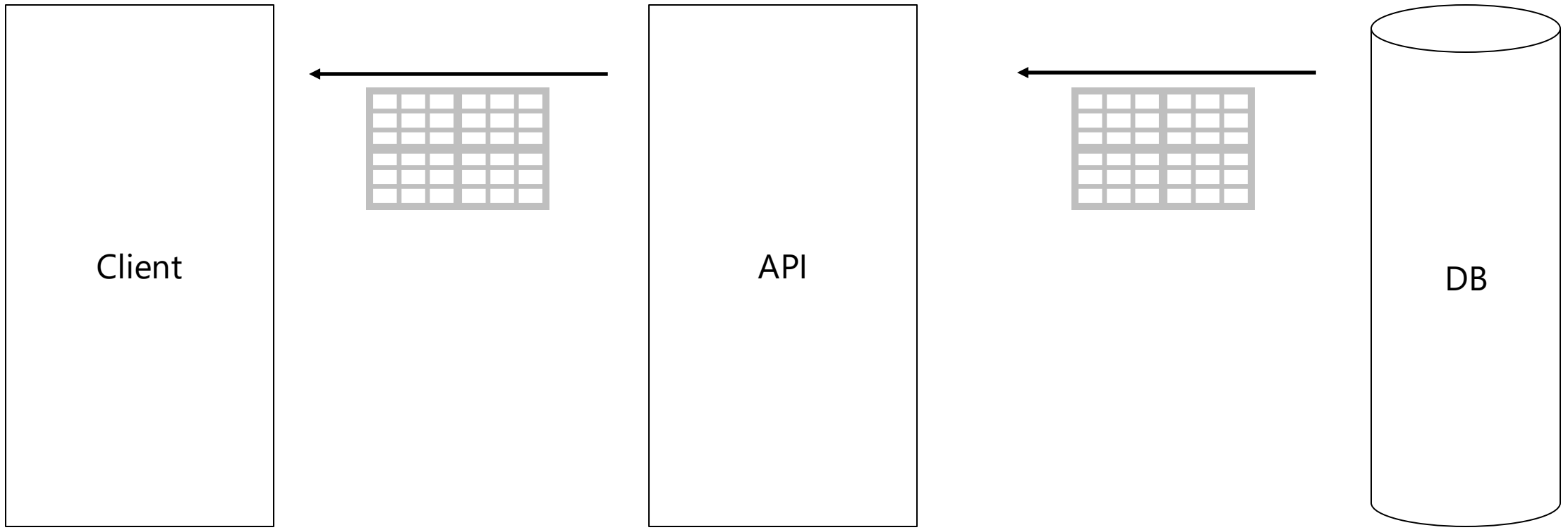
Use DTOs / ViewModels



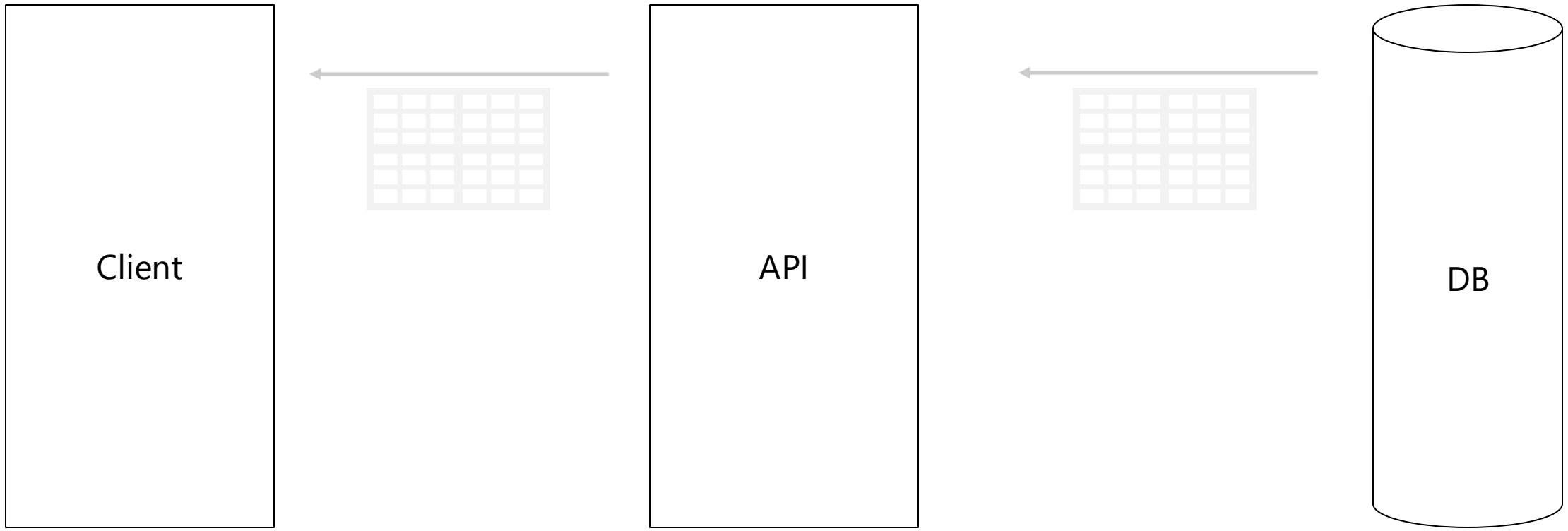
Use DTOs / ViewModels



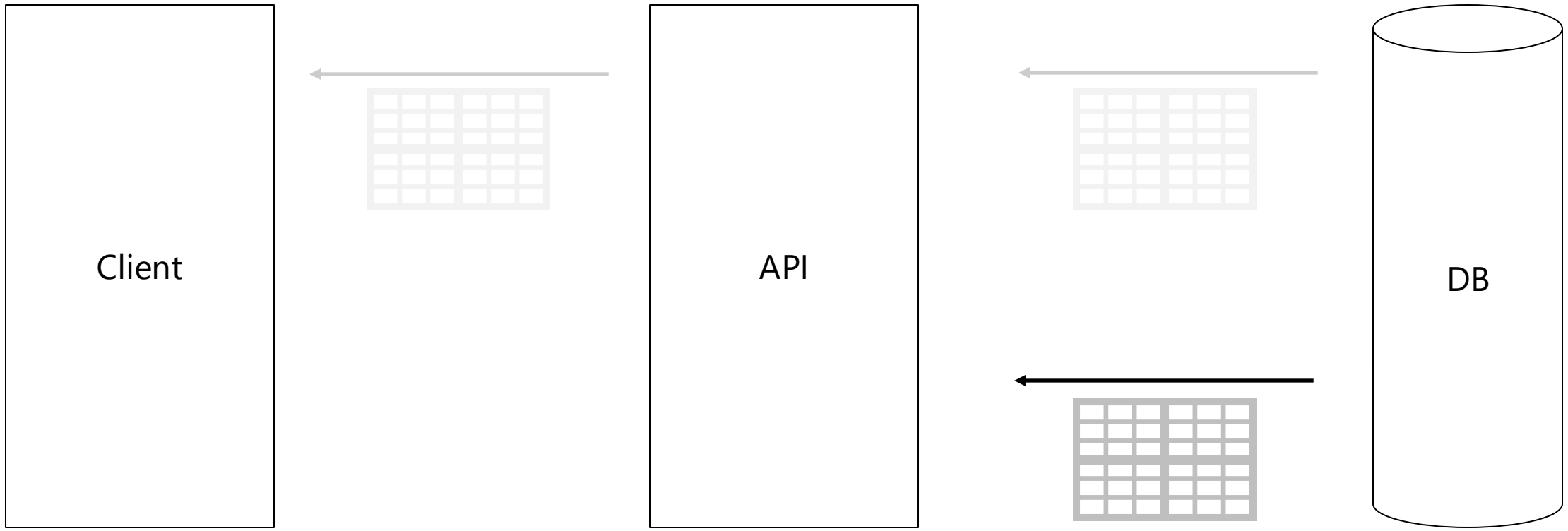
Use DTOs / ViewModels



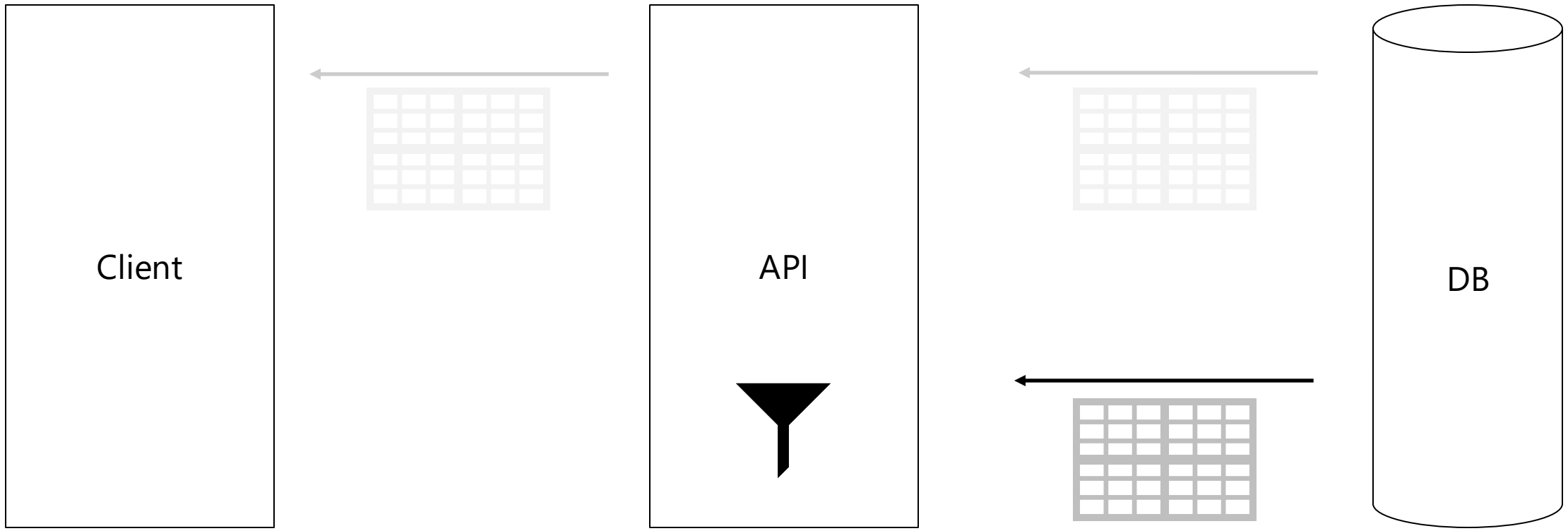
Use DTOs / ViewModels



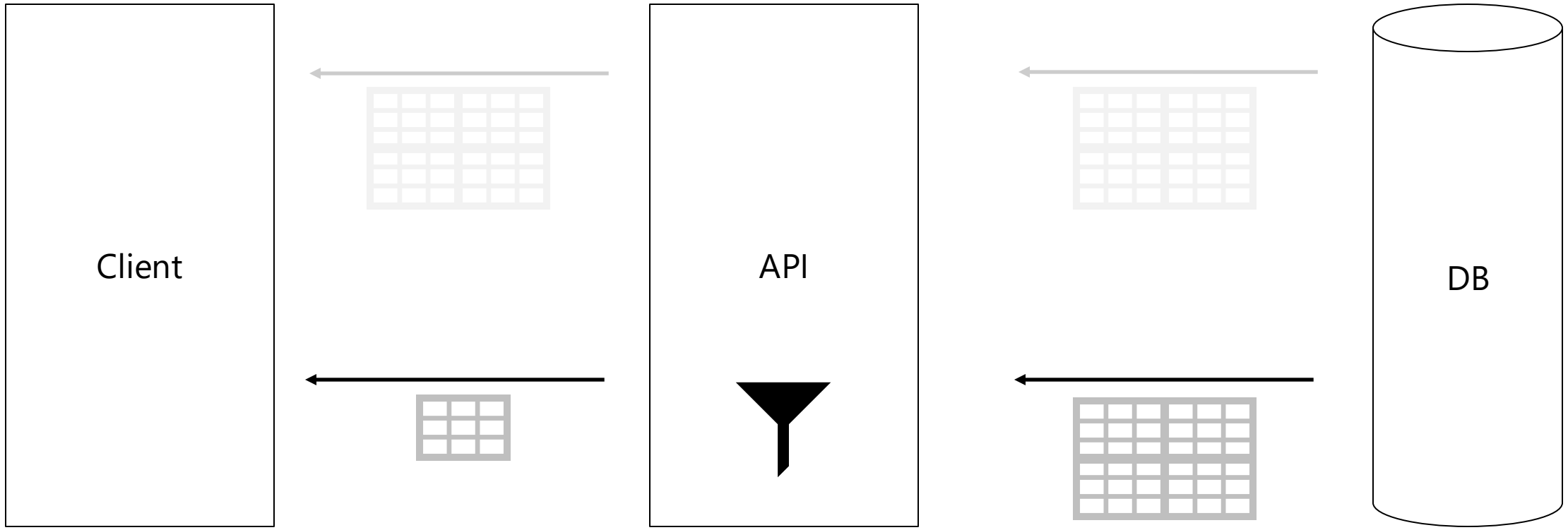
Use DTOs / ViewModels



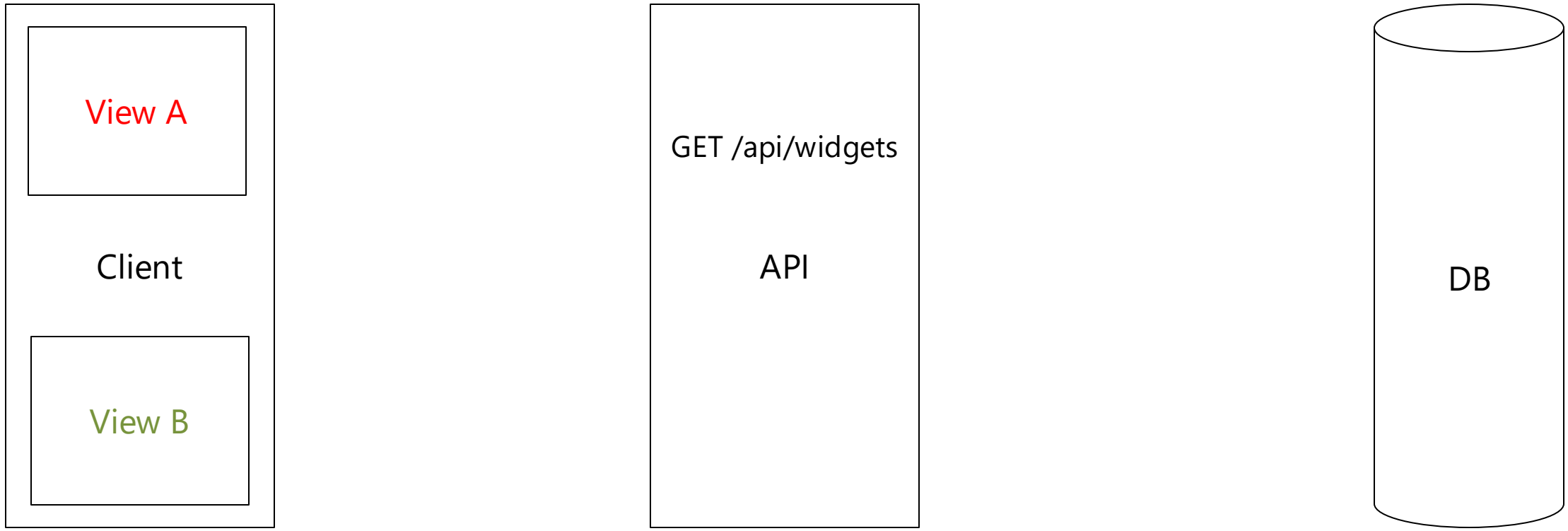
Use DTOs / ViewModels



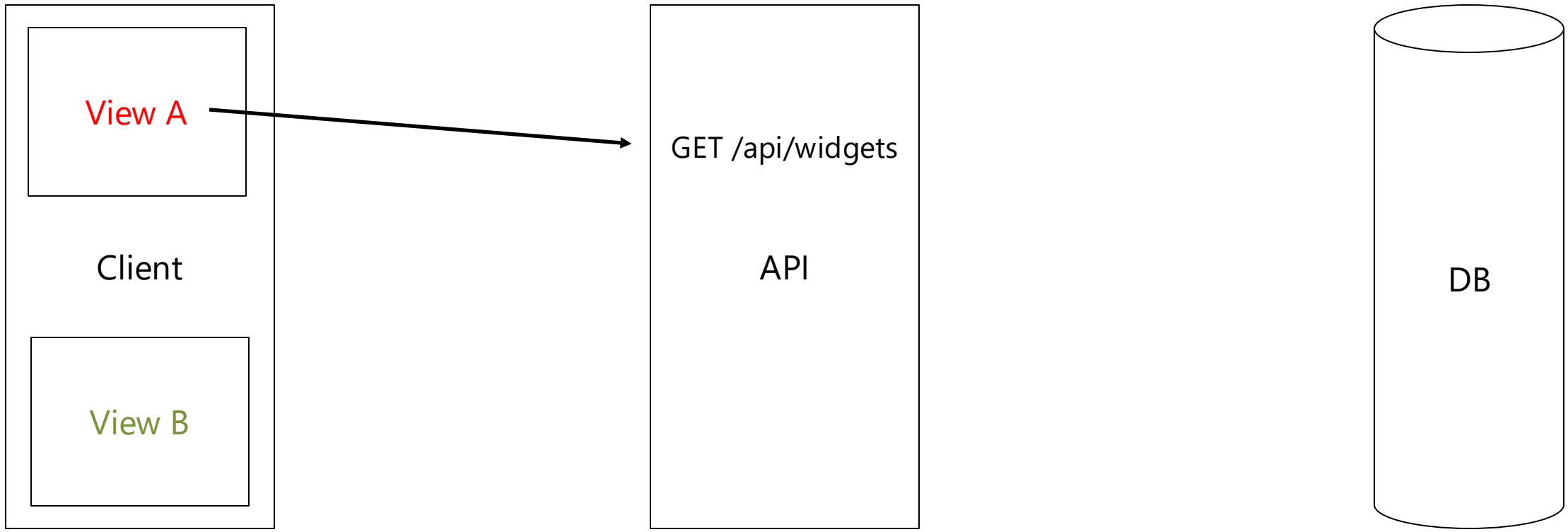
Use DTOs / ViewModels



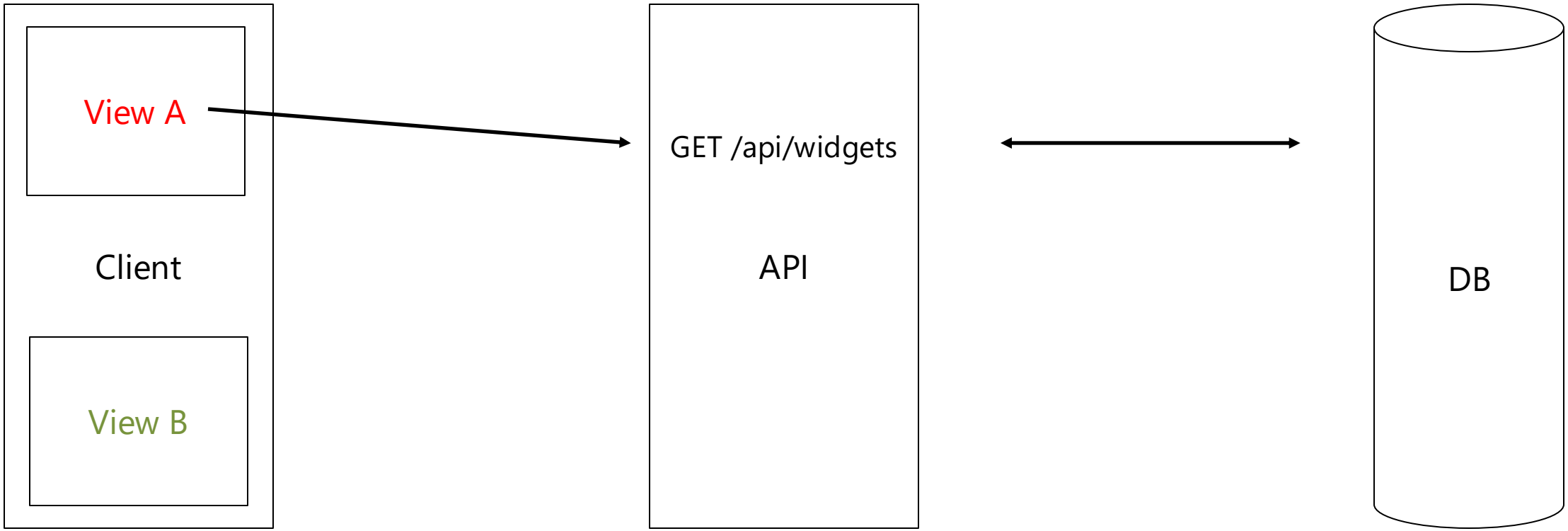
Avoid General APIs



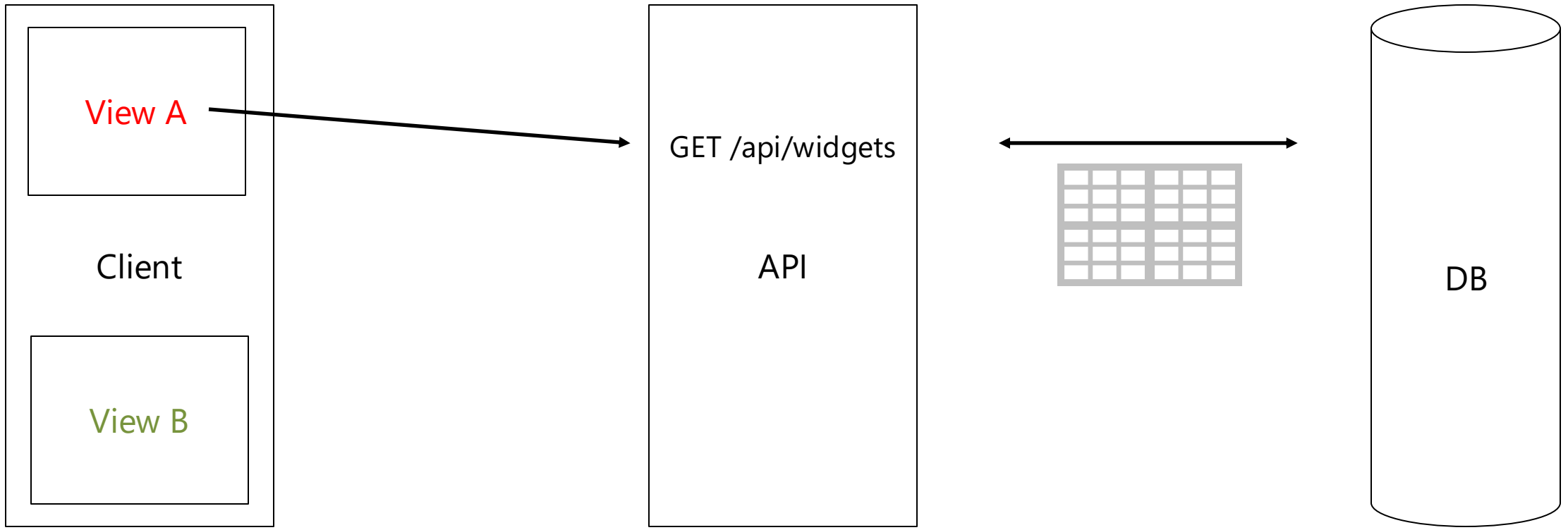
Avoid General APIs



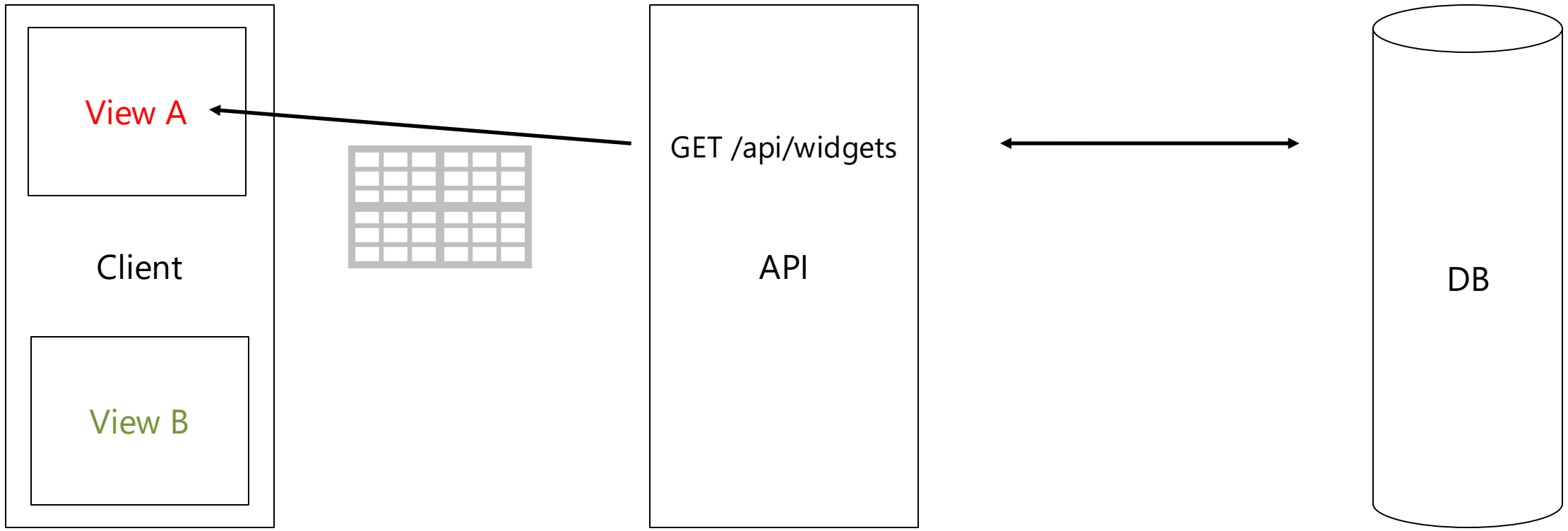
Avoid General APIs



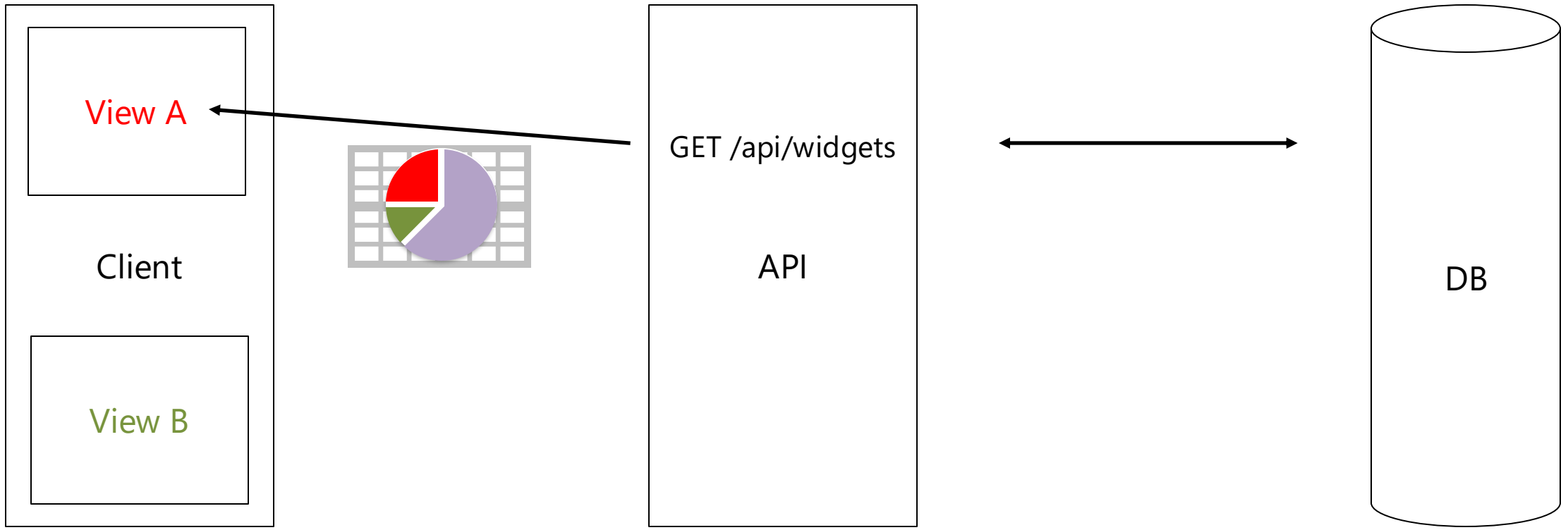
Avoid General APIs



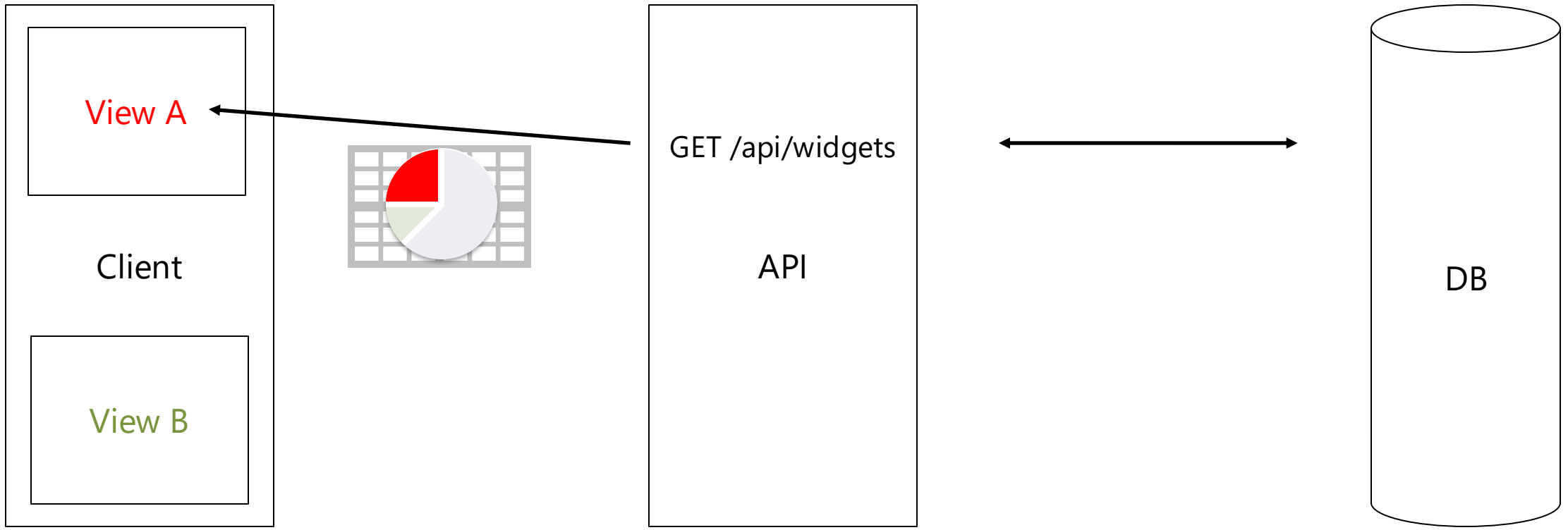
Avoid General APIs



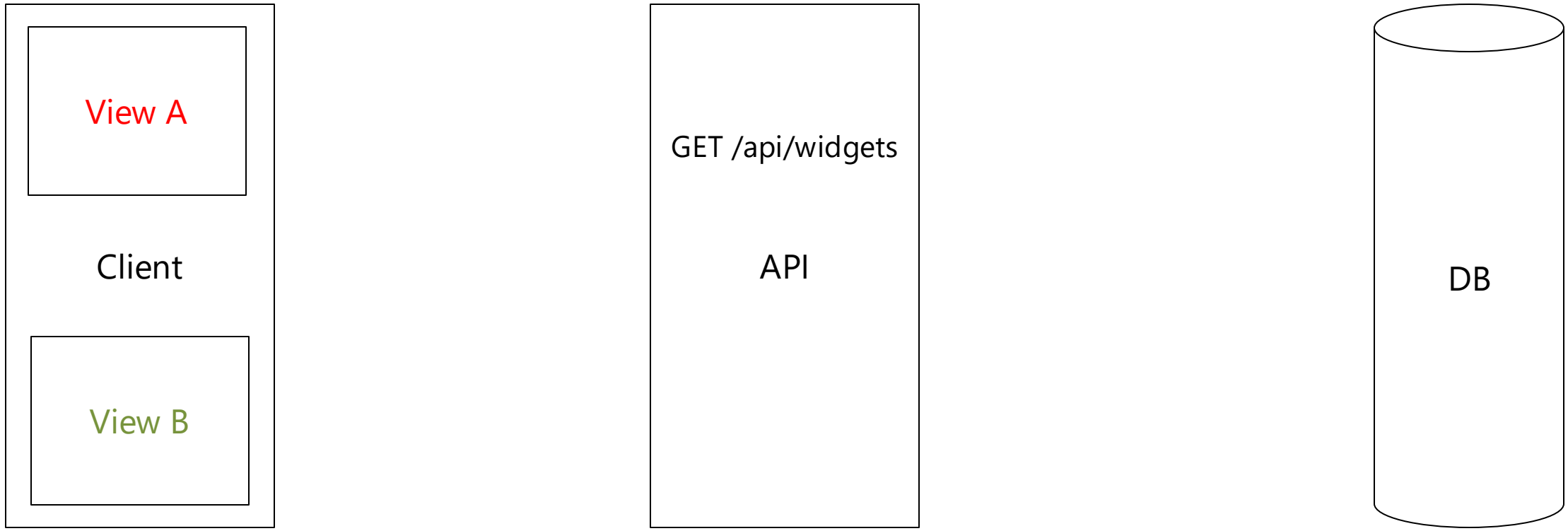
Avoid General APIs



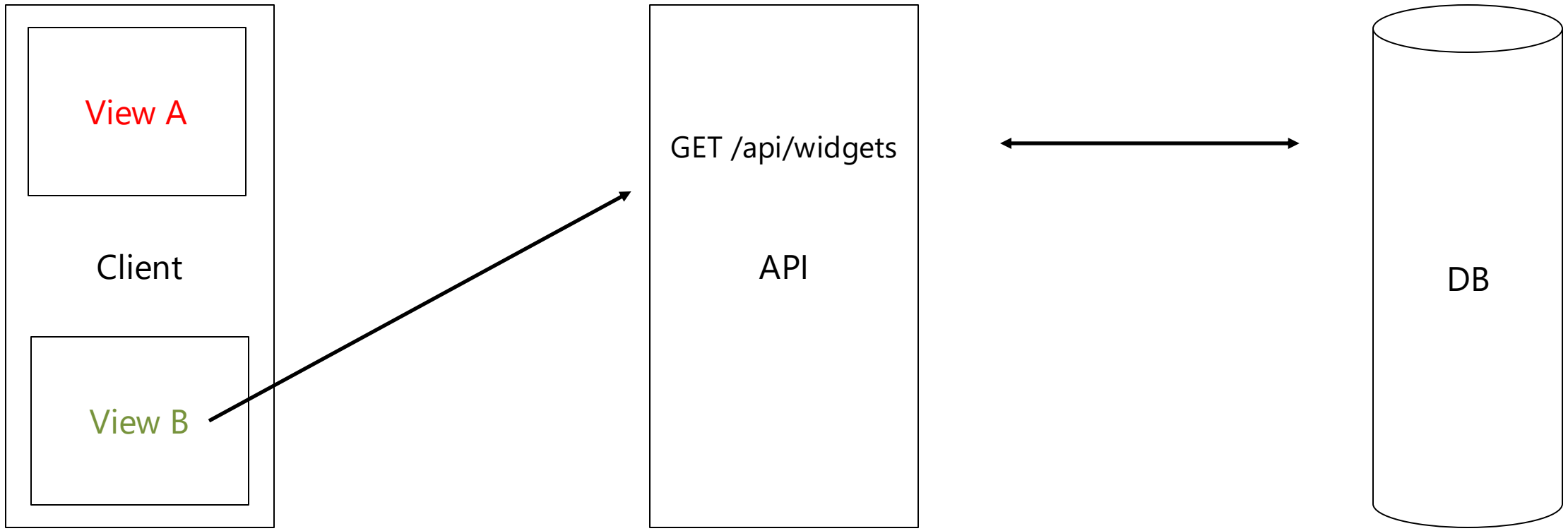
Avoid General APIs



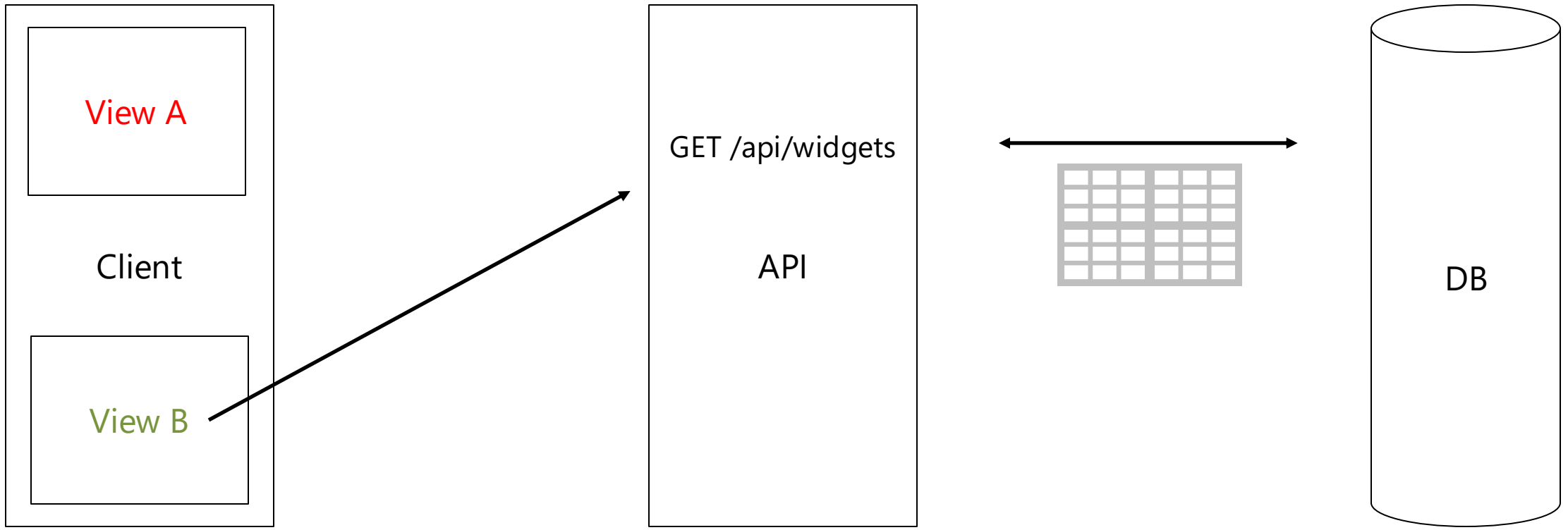
Avoid General APIs



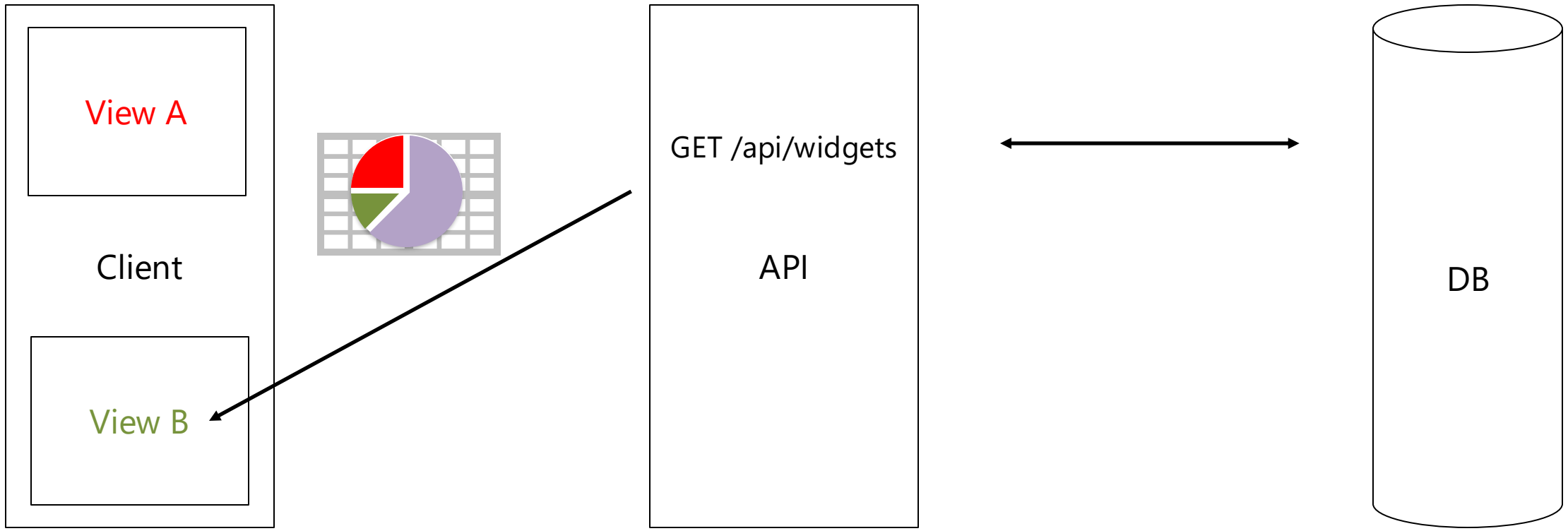
Avoid General APIs



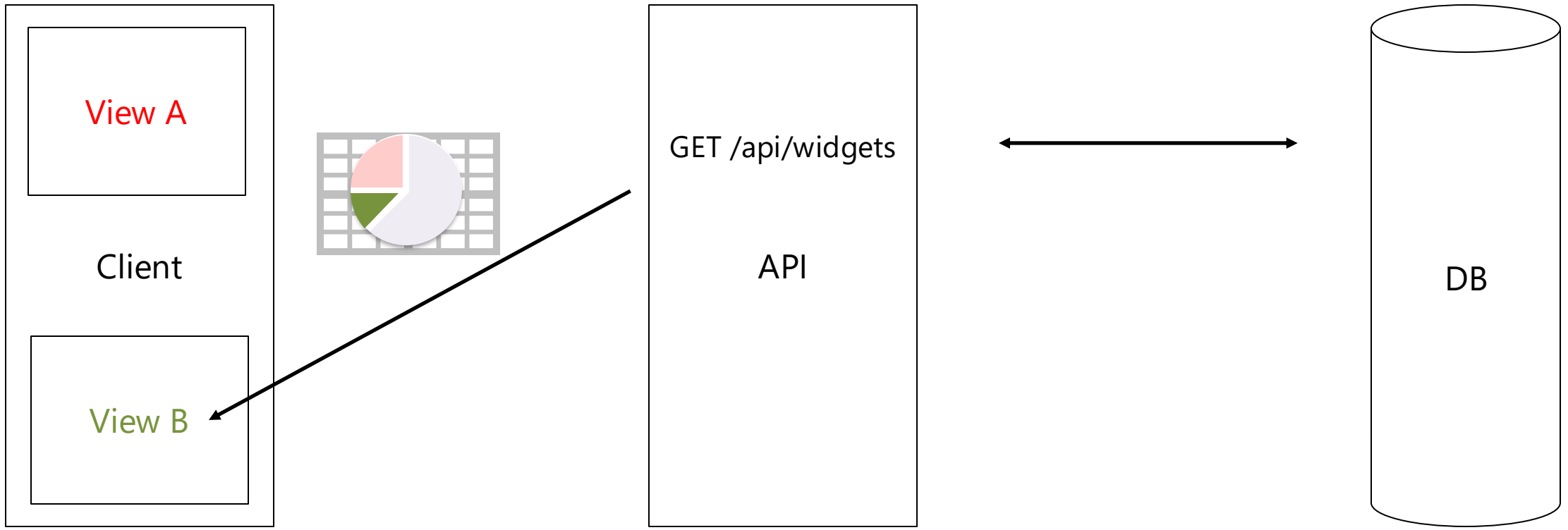
Avoid General APIs



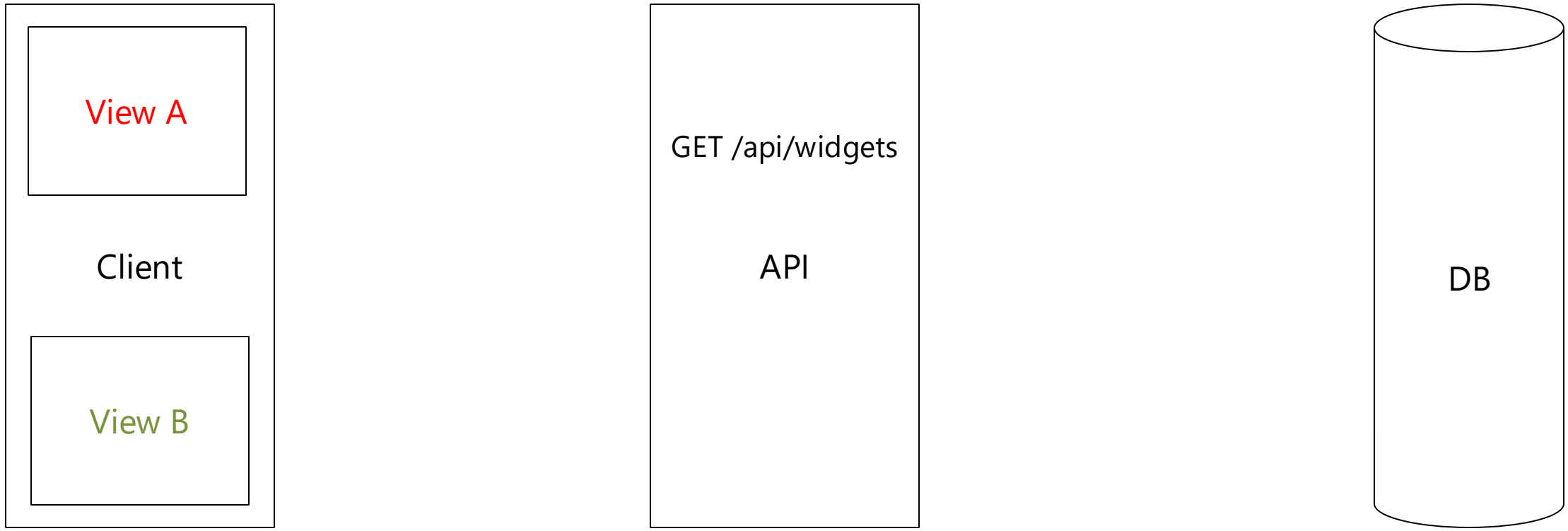
Avoid General APIs



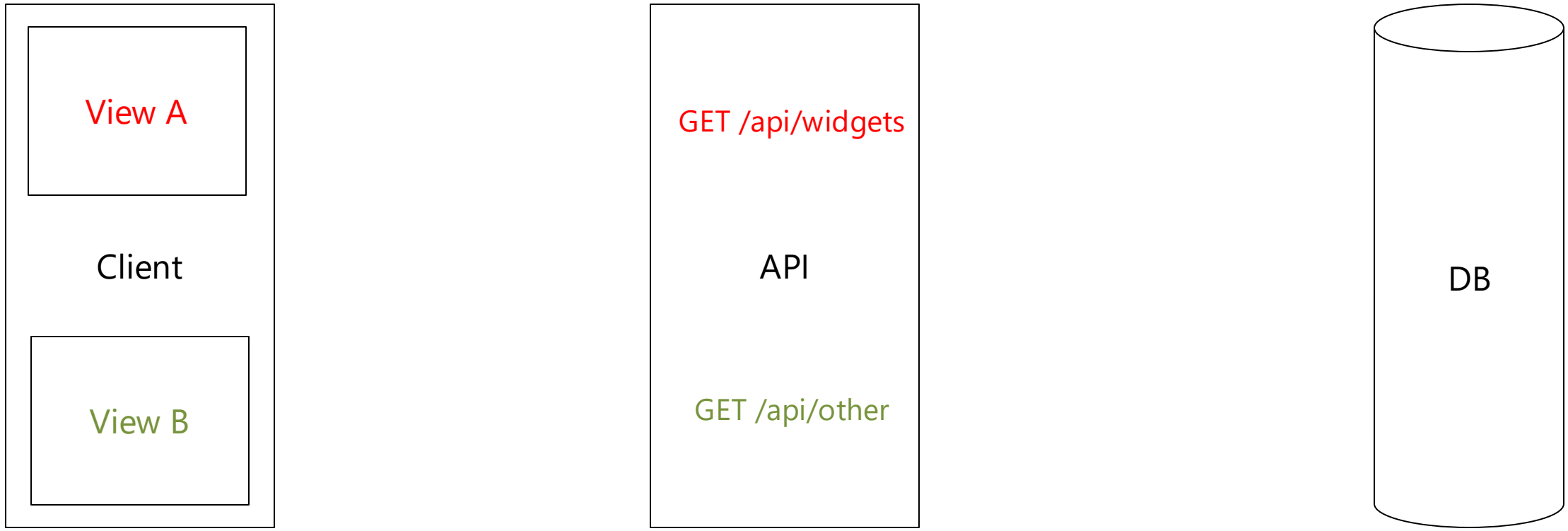
Avoid General APIs



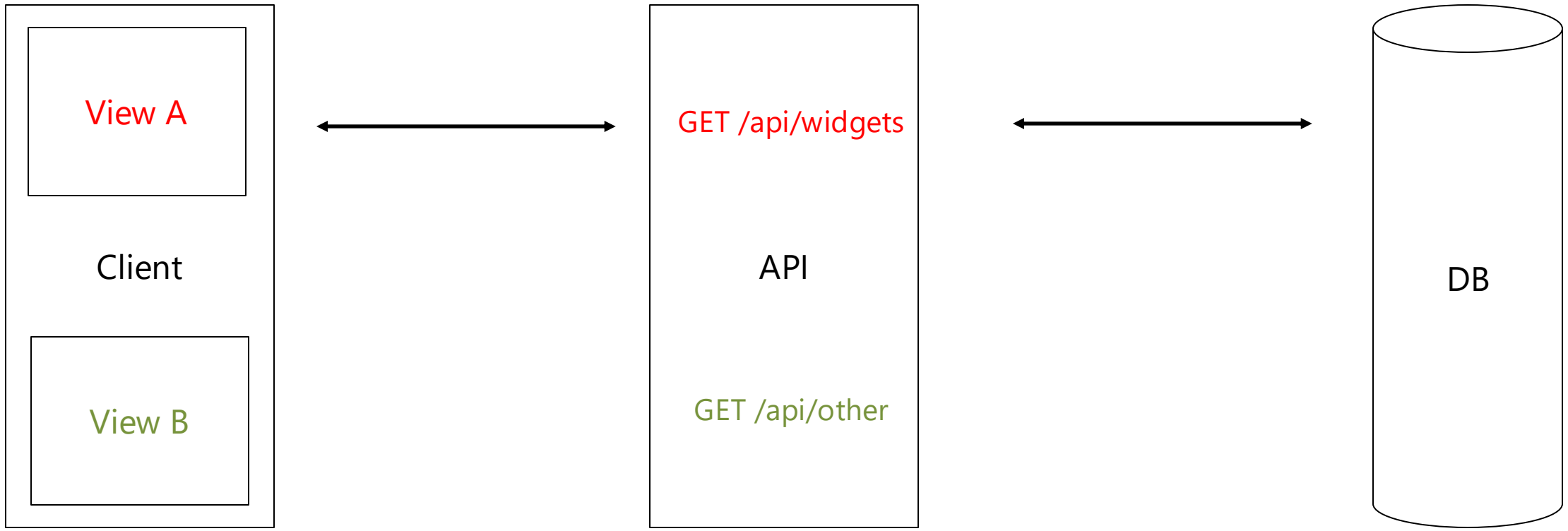
Avoid General APIs



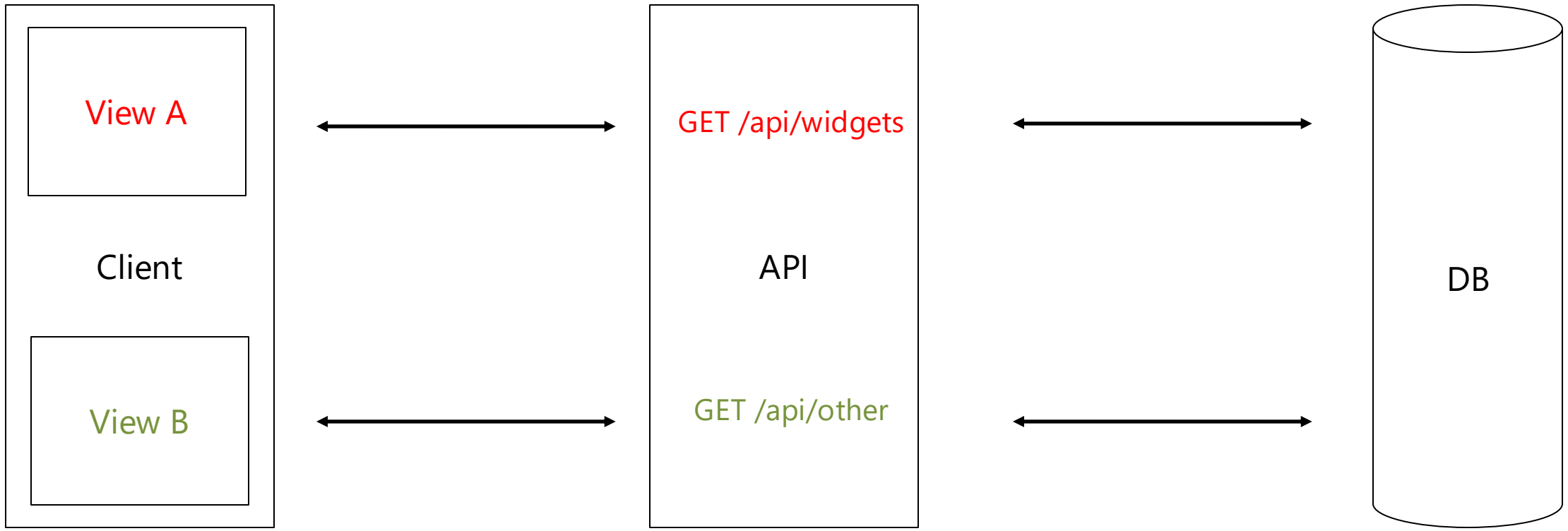
Avoid General APIs



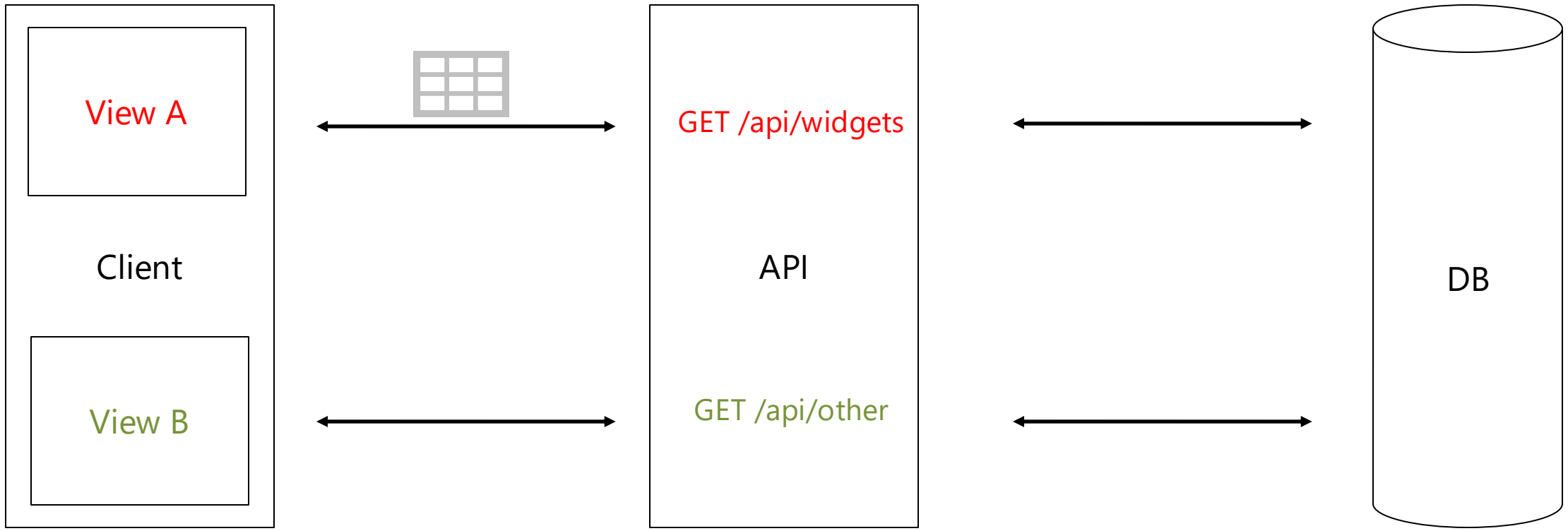
Avoid General APIs



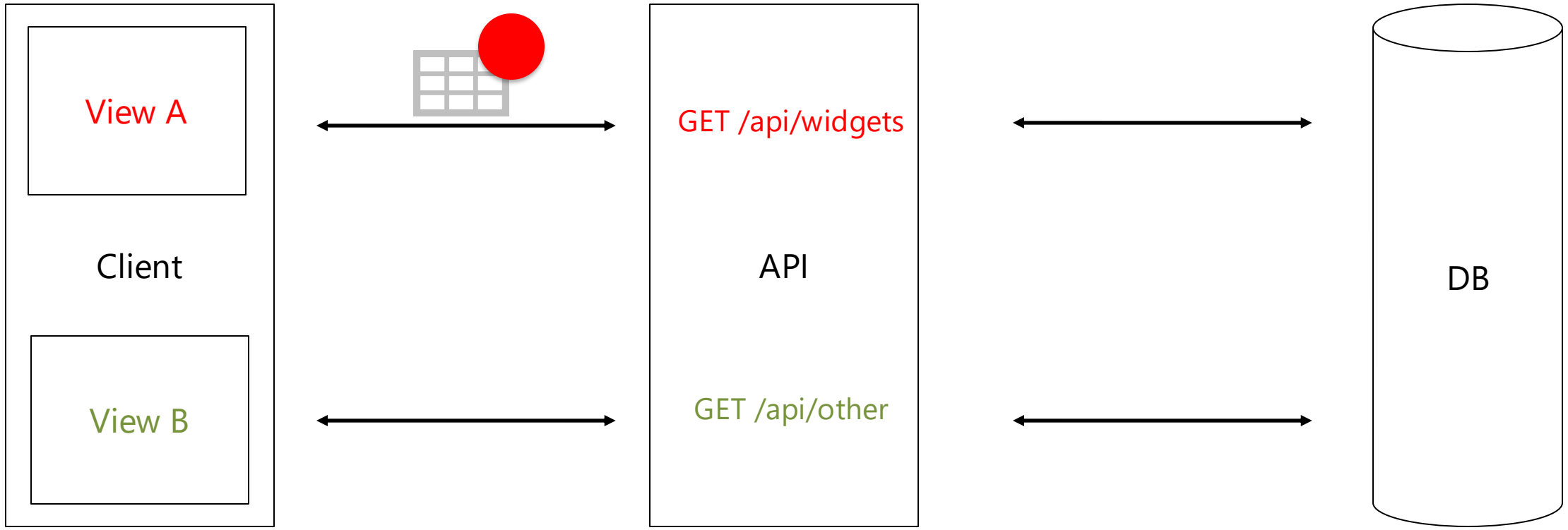
Avoid General APIs



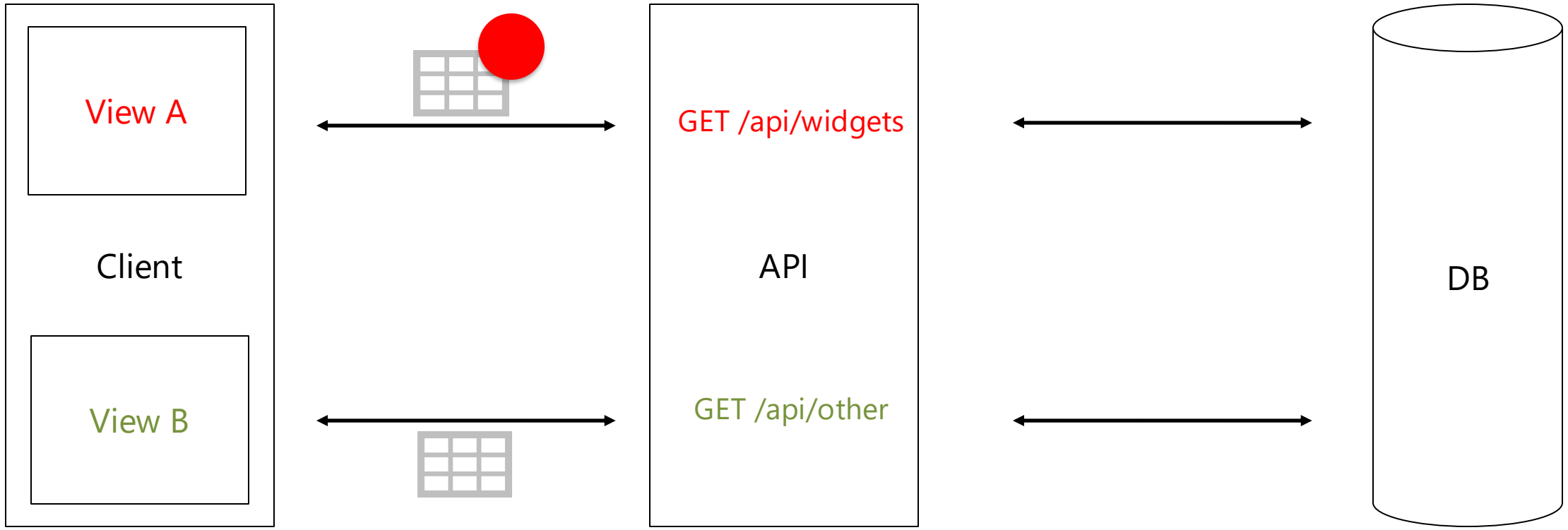
Avoid General APIs



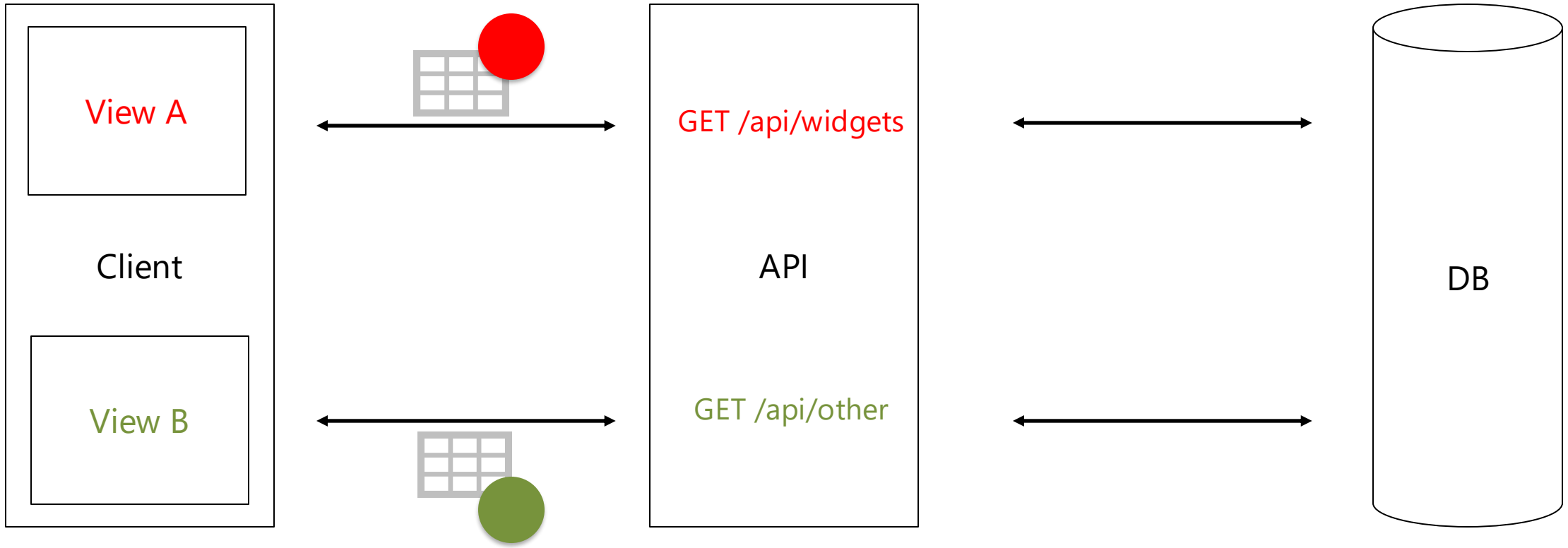
Avoid General APIs



Avoid General APIs

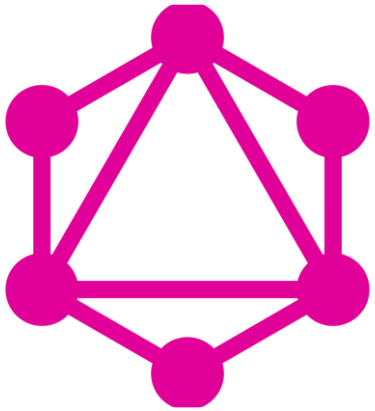


Avoid General APIs



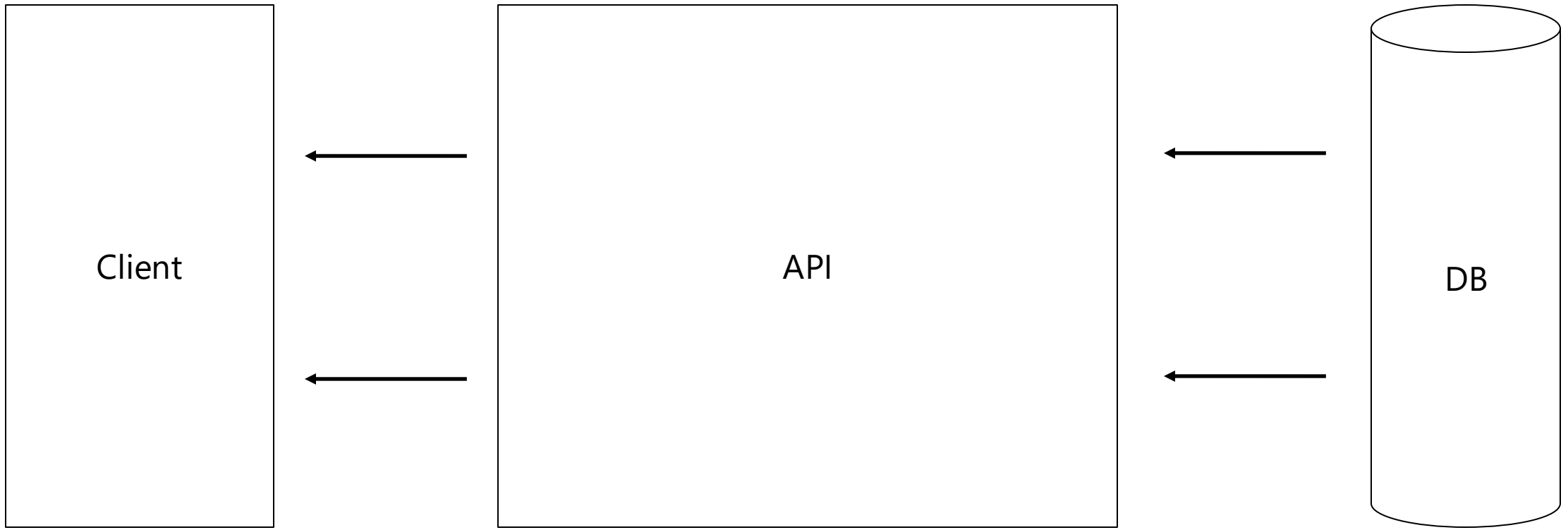
Avoid General APIs

OData

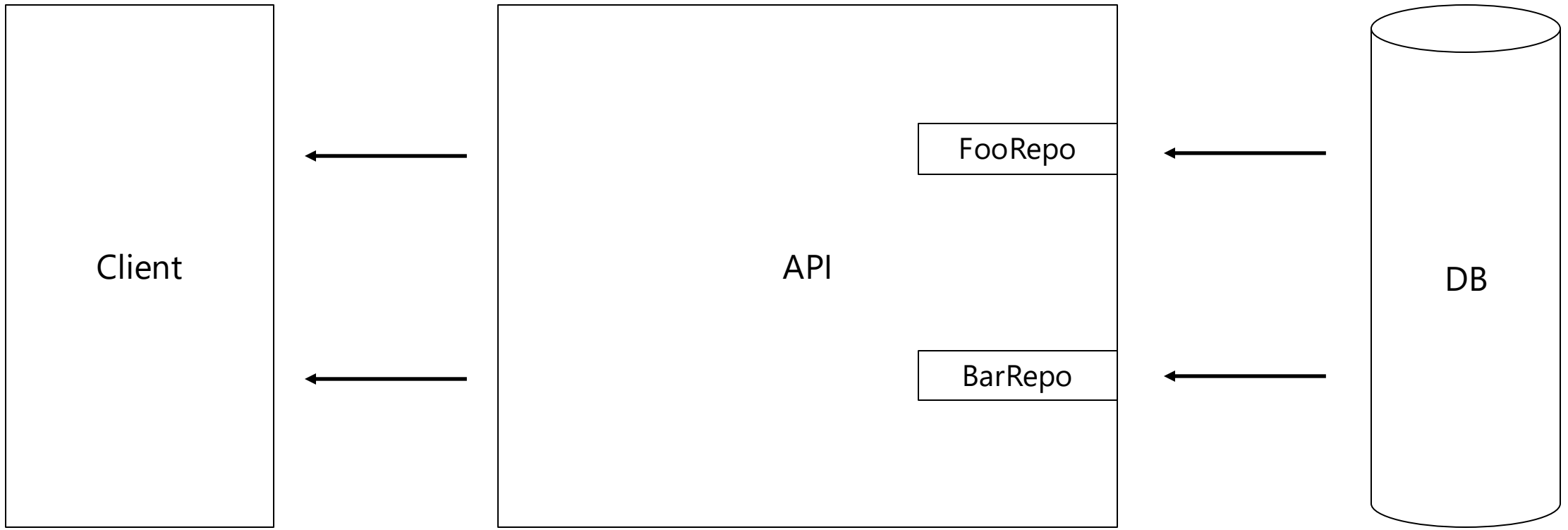


GraphQL

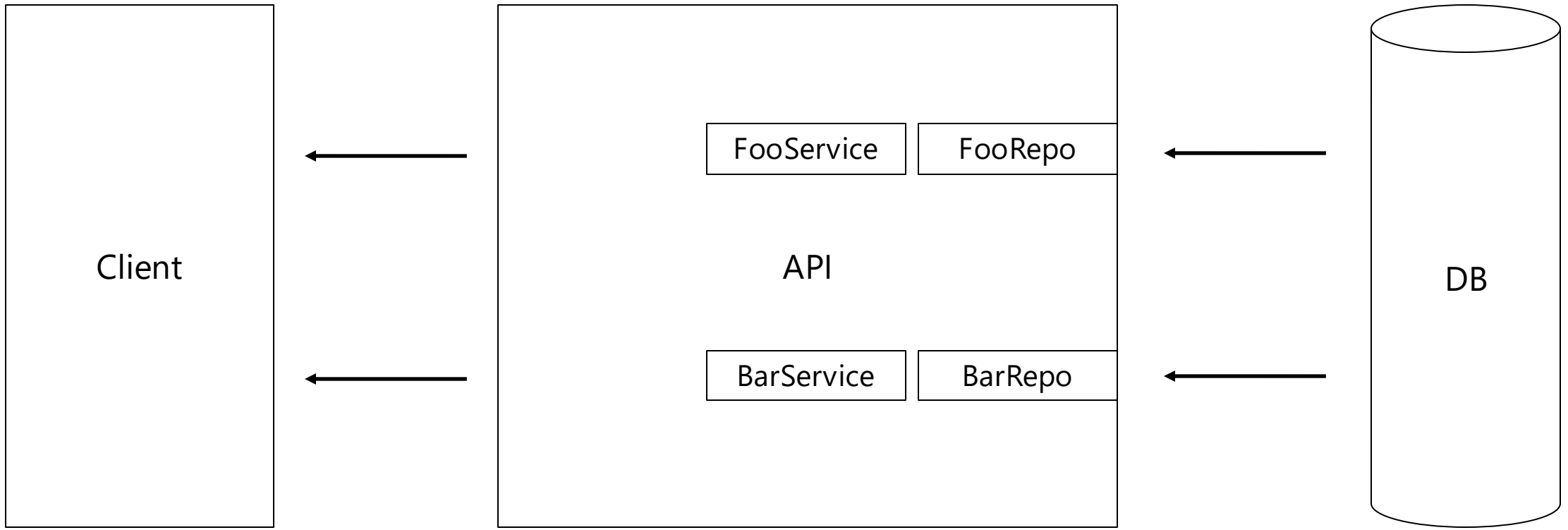
Avoid General APIs



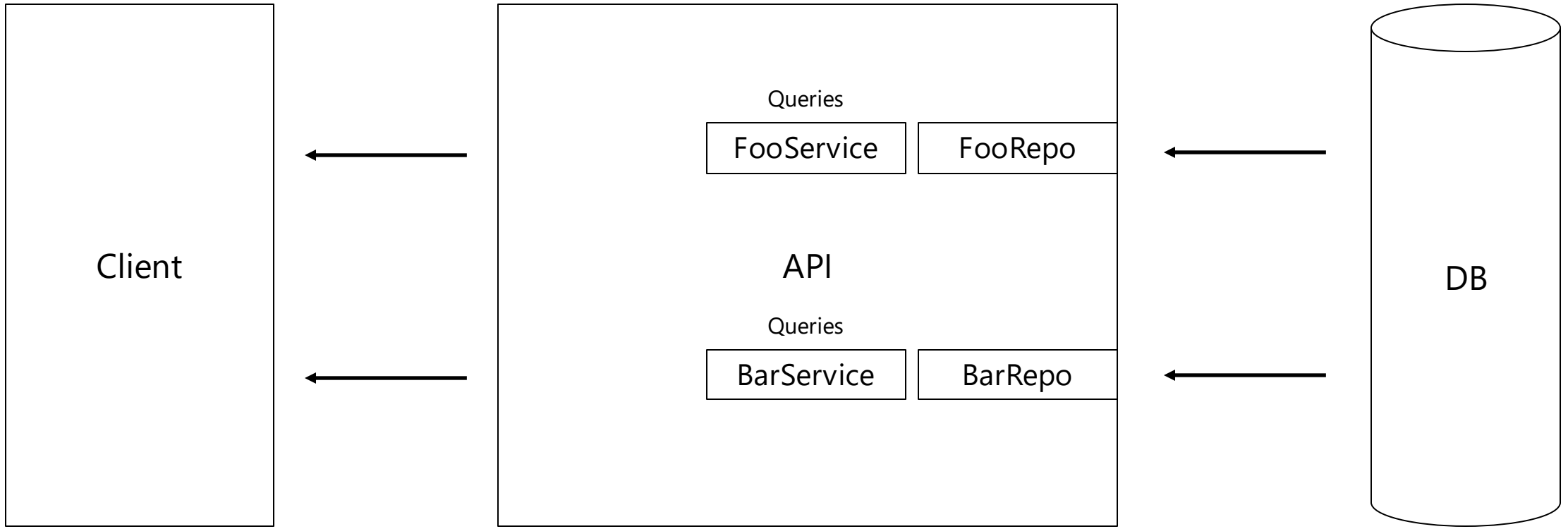
Avoid General APIs



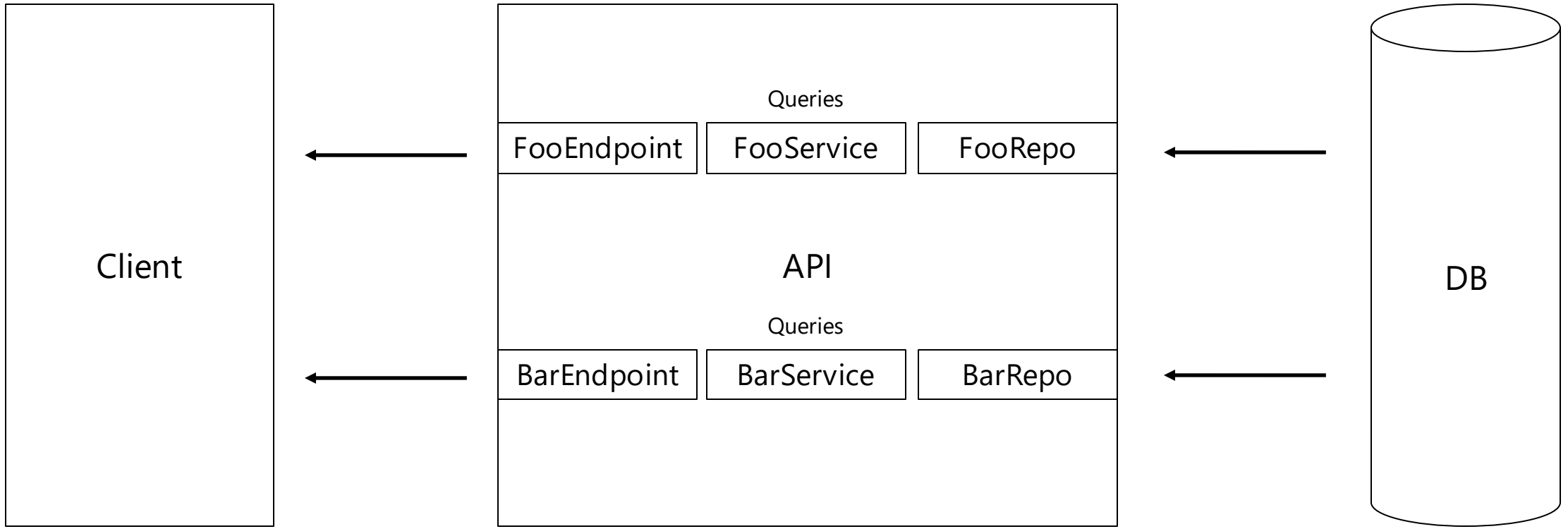
Avoid General APIs



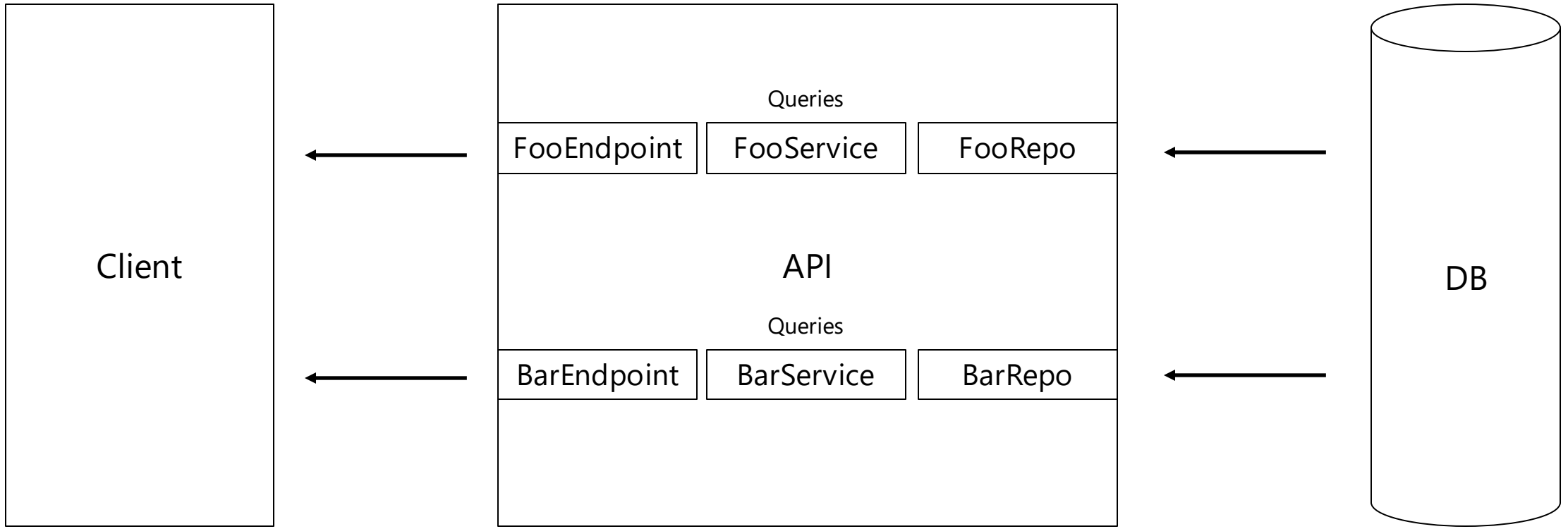
Avoid General APIs



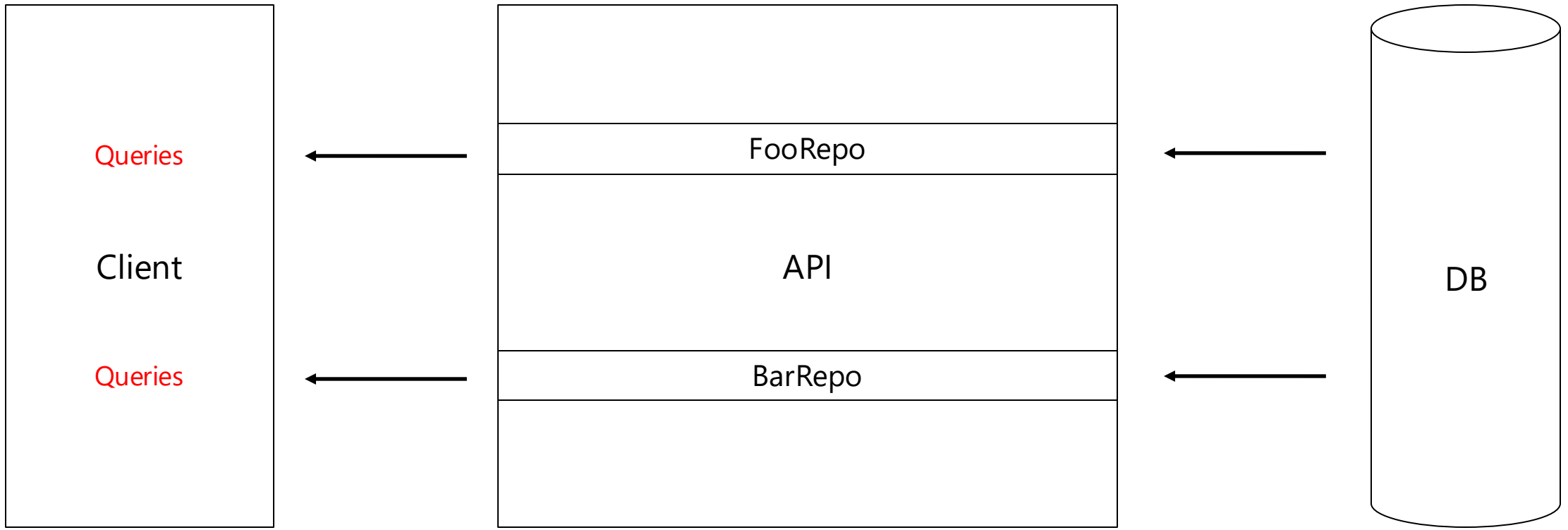
Avoid General APIs



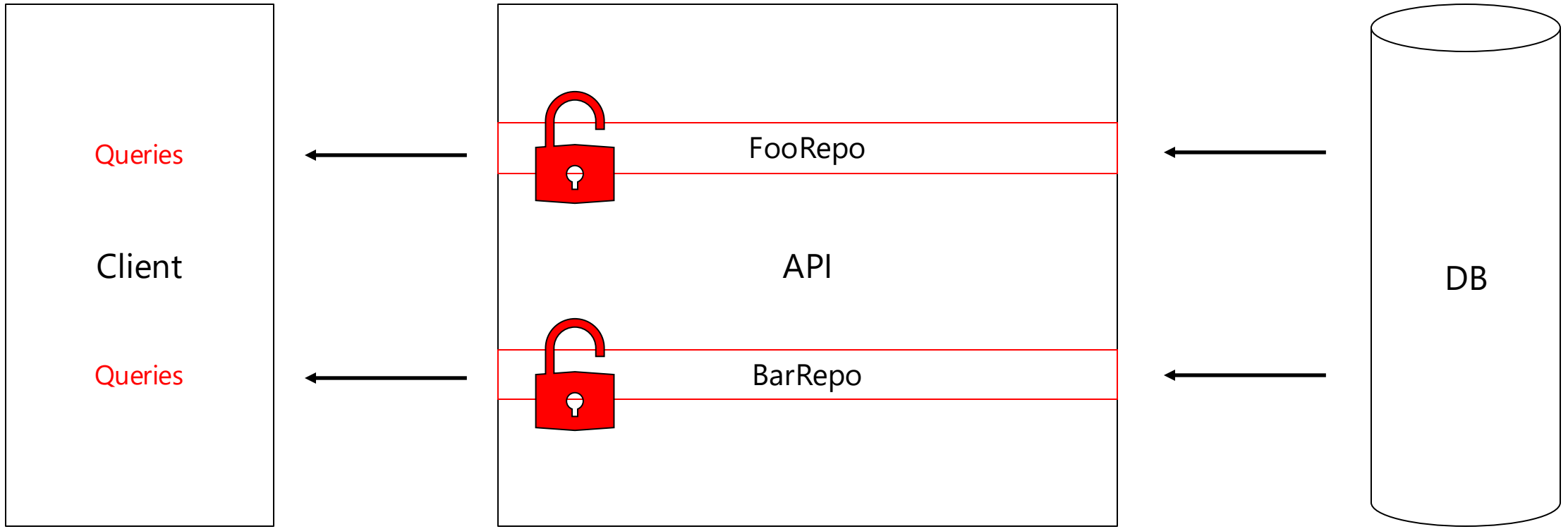
Avoid General APIs



Avoid General APIs



Avoid General APIs



Large Data Handling

```
{
  ...
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA...”
  ...
}
```



```
{
  ...
  "profilePicture": "https//img.domain.com/4AF980...”
  ...
}
```

Large Data Handling

```
{
  ...
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."
  ...
}
```

```
{
  ...
  "profilePictureUri": "https//img.domain.com/4F0..."
  ...
}
```

Broken Object Property Level Authorization

Common Mistake #3

TRAILHEAD
TECHNOLOGY PARTNERS

Broken Authentication

Common Mistake #4

TRAILHEAD
TECHNOLOGY PARTNERS

Weak Signing Key

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ...
            IssuerSigningKey = "secret123", // ❌ weak
            ...
        };
    });
```

Core Token Checks Disabled

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(o =>
{
    o.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = false,           // ❌ off
        ValidateAudience = false,         // ❌ off
        ValidateLifetime = false,         // ❌ off
        ValidateIssuerSigningKey = false // ❌ key not checked
        // (no ValidAlgorithms set)
    };
});
```

Mixed Environment Configuration

Environment	SigningKey	Audience	Issuer
Development	NmQ5NjI2YWYtMDUyNi00MmL…	AppUser	AppName
Testing	NmQ5NjI2YWYtMDUyNi00MmL…	AppUser	AppName
Staging	NmQ5NjI2YWYtMDUyNi00MmL…	AppUser	AppName
Production	NmQ5NjI2YWYtMDUyNi00MmL…	AppUser	AppName

Mixed Environment Configuration

Environment	SigningKey	Audience	Issuer
Development	NmQ5NjI2YWYtMDUyNi00MmL…	AppUser-dev	https://login-dev.appname.local
Testing	YWViYThiYzItZDI3Ny00MzUzLFk…	AppUser-test	https://login-test.appname.internal
Staging	ZTQyMmM1ZWUtNTNlNy00ZjL…	AppUser-staging	https://login-staging.appname.com
Production	OTdiNDdmMjI5NGFlNDAxNDMT…	AppUser	https://login.appname.com

Long-lived Access Tokens

Access tokens last hours or days

Refresh tokens never rotate

Long-lived Access Tokens

Short access token TTL (5–15 min)

Rotate refresh tokens and **revoke the old** on use

Invalidate all tokens on password reset

Insecure Cookies 

```
.AddCookie(o =>
{
    o.Cookie.Name = ".app.auth";
    o.Cookie.HttpOnly = false;                          // ❌ JS can read cookie
    o.Cookie.SecurePolicy = CookieSecurePolicy.None; // ❌ sent over HTTP
    o.Cookie.SameSite = SameSiteMode.None;              // ❌ CSRF risk if not Secure
    o.ExpireTimeSpan = TimeSpan.FromDays(30);           // ❌ very long-lived
    o.SlidingExpiration = false;                        // ❌ no refresh-on-use
});
```

Insecure Cookies 

```
.AddCookie(o =>
{
    o.Cookie.Name = ".app.auth";
    o.Cookie.HttpOnly = true;                                  // ✅ not readable by JS
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always; // ✅ HTTPS only
    o.Cookie.SameSite = SameSiteMode.Lax;                 // ✅ mitigates CSRF
    o.ExpireTimeSpan = TimeSpan.FromMinutes(30);     // ✅ short-lived
    o.SlidingExpiration = true;                              // ✅ extend on active use
    o.Events = new CookieAuthenticationEvents
    {
        // Optional: add extra validation, e.g., check revoked sessions
    };
});
```

Misconfiguration and Insecure Defaults

Common Mistake #5

TRAILHEAD
TECHNOLOGY PARTNERS

Security Headers 

```
var builder = WebApplication.CreateBuilder(args); // default kestrel config
```

Security Headers

```
builder.WebHost.ConfigureKestrel(o => o.AddServerHeader = false);
app.Use(async (ctx, next) =>
{
  ctx.Response.Headers["X-Content-Type-Options"] = "nosniff";
  ctx.Response.Headers["Referrer-Policy"] = "no-referrer";
  ctx.Response.Headers["Permissions-Policy"] = "geolocation=()";
  await next();
});
```

Proxy and Load Balancer Awareness

Behind a proxy but not forwarding scheme or host.

Proxy and Load Balancer Awareness

```
using Microsoft.AspNetCore.HttpOverrides;
app.UseForwardedHeaders(new ForwardedHeadersOptions
{
  ForwardedHeaders = ForwardedHeaders.XForwardedFor |
                     ForwardedHeaders.XForwardedProto |
                     ForwardedHeaders.XForwardedHost,
  KnownProxies = { System.Net.IPAddress.Parse("10.0.0.5") }
});
```

CORS That Is Not Wide Open

```
builder.Services.AddCors(p => p.AddPolicy("open", c => c
  .AllowAnyOrigin()
  .AllowAnyHeader()
  .AllowAnyMethod()
  .AllowCredentials()));
```

CORS That Is Not Wide Open

```
builder.Services.AddCors(p => p.AddPolicy("spa", c => c
  .WithOrigins("https://app.example.com")
  .WithHeaders("Content-Type","Authorization")
  .WithMethods("GET","POST","PUT","DELETE")
  .AllowCredentials()));
app.UseCors("spa");
```

OpenAPI in Production

```
app.UseSwagger();
app.UseSwaggerUI();
app.MapHealthChecks("/healthz"); // simple liveness
```

OpenAPI in Production

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapSwagger().RequireAuthorization("Docs.Read");
app.MapHealthChecks("/healthz"); // simple liveness
```

Cookies 

```
.AddCookie(o => {
  o.Cookie.HttpOnly = false;
  o.Cookie.SecurePolicy = CookieSecurePolicy.None;
  o.Cookie.SameSite = SameSiteMode.None;
});
```

Cookies 

```
.AddCookie(o => {
  o.Cookie.HttpOnly = true;
  o.Cookie.SecurePolicy = CookieSecurePolicy.Always;
  o.Cookie.SameSite = SameSiteMode.Lax; // Strict if same-site
  o.SlidingExpiration = true;
});
```

JSON Binding and Response Shaping

Bind entities directly

Unknown JSON fields ignored

JSON Binding and Response Shaping

```
services.AddControllers().AddJsonOptions(o =>
  o.JsonSerializerOptions.UnmappedMemberHandling =
    System.Text.Json.Serialization.JsonUnmappedMemberHandling.Disallow);
```

Unrestricted Resource Consumption

Common Mistake #6

Paging and Query Shape

```
// Unbounded read: client controls page size
[HttpGet] public Task<List<Order>> Get(int page = 1, int pageSize = 5000)
    => db.Orders.Skip((page-1)*pageSize).Take(pageSize).ToListAsync();
```

Paging and Query Shape

```
const int MaxPageSize = 100; // server-enforced cap
[HttpGet] public async Task<IResult> Get(int page = 1, int pageSize = 50,
CancellationToken ct = default)
{
    pageSize = Math.Clamp(pageSize, 1, MaxPageSize);
    var items = await db.Orders
        .AsNoTracking()
        .OrderByDescending(o => o.CreatedUtc)
        .Select(o => new OrderDto { Id = o.Id, Total = o.Total })
        .Skip((page-1)*pageSize).Take(pageSize)
        .ToListAsync(ct);
    return Results.Ok(items);
}
```

Rate Limiting 

```
// No throttling; login and search endpoints can be hammered
app.MapPost("/login", LoginHandler);
app.MapGet("/search", SearchHandler);
```

Rate Limiting

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = 429;
    o.AddPolicy("tight", ctx => RateLimitPartition.GetTokenBucketLimiter(
        ctx.Connection.RemoteIpAddress?.ToString() ?? "anon",
        _ => new TokenBucketRateLimiterOptions
        {
            TokenLimit = 5,            // max in bucket
            TokensPerPeriod = 5,       // refill each period
            ReplenishmentPeriod = TimeSpan.FromMinutes(1),
            AutoReplenishment = true
        }));
});

...

// Apply the single policy to just the sensitive endpoint(s)
app.UseRateLimiter();
app.MapPost("/login", LoginHandler).RequireRateLimiting("tight");
app.MapGroup("/auth").RequireRateLimiting("tight"); // apply to many at once
```

Request Body Size

Kestrel Default is 30 MB

Request Body Size

```
builder.WebHost.ConfigureKestrel(o =>
    o.Limits.MaxRequestBodySize = 2 * 1024 * 1024); // 2 MB
```

Verbose Errors & Logging Leaks

Common Mistake #7

TRAILHEAD
TECHNOLOGY PARTNERS

Error Handling in Production

```
app.UseDeveloperExceptionPage();
```

Error Handling in Production

```
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/error");

    app.Map("/error", () => Results.Problem(
        statusCode: 500,
        title: "An error occurred"));  // keep generic
}
```

ProblemDetails, Not Stack Traces 

```
// ❌ Shows stack traces in every environment
app.UseDeveloperExceptionPage();

app.MapGet("/boom", () => throw new InvalidOperationException("DB timeout..."));

app.Run();
```

ProblemDetails, Not Stack Traces 

```
app.MapGet("/boom", () => throw new InvalidOperationException("DB timeout..."));

// ✅ ProblemDetails endpoint (generic, no stack trace)
app.Map("/error", (HttpContext ctx) =>
{
    // Optional: attach a correlation id for support
    var cid = ctx.TraceIdentifier;
    return Results.Problem(
        statusCode: StatusCodes.Status500InternalServerError,
        title: "An error occurred",
        extensions: new Dictionary<string, object?> { ["correlationId"] = cid }
    );
});

app.Run();
```

Redact Secrets In Logs 

```
builder.Services.AddHttpLogging(); // ✗ logs lots of headers/body by default

builder.Logging.AddConsole();
builder.Logging.SetMinimumLevel(LogLevel.Information);

optionsBuilder
    .EnableSensitiveDataLogging()  // ✗ logs PII and secrets in SQL params
    .EnableDetailedErrors()
    .LogTo(Console.WriteLine, LogLevel.Information); // ✗ broad, unfiltered
```

Redact Secrets In Logs 

```
// HTTP logging: safe fields only
b.Services.AddHttpLogging(o =>
    o.LoggingFields = HttpLoggingFields.RequestPath |
                      HttpLoggingFields.RequestMethod |
                      HttpLoggingFields.ResponseStatusCode);

// EF Core: no sensitive data logging; show SQL shape in Dev only
b.Services.AddDbContext<AppDbContext>(opts =>
{
    if (b.Environment.IsDevelopment())
        opts.EnableDetailedErrors()
            .LogTo(Console.WriteLine,
                   new[] { DbLoggerCategory.Database.Command.Name },
                   LogLevel.Information);
    // Prod: default (no LogTo, no SensitiveDataLogging)
});
```

Redact Secrets In Logs

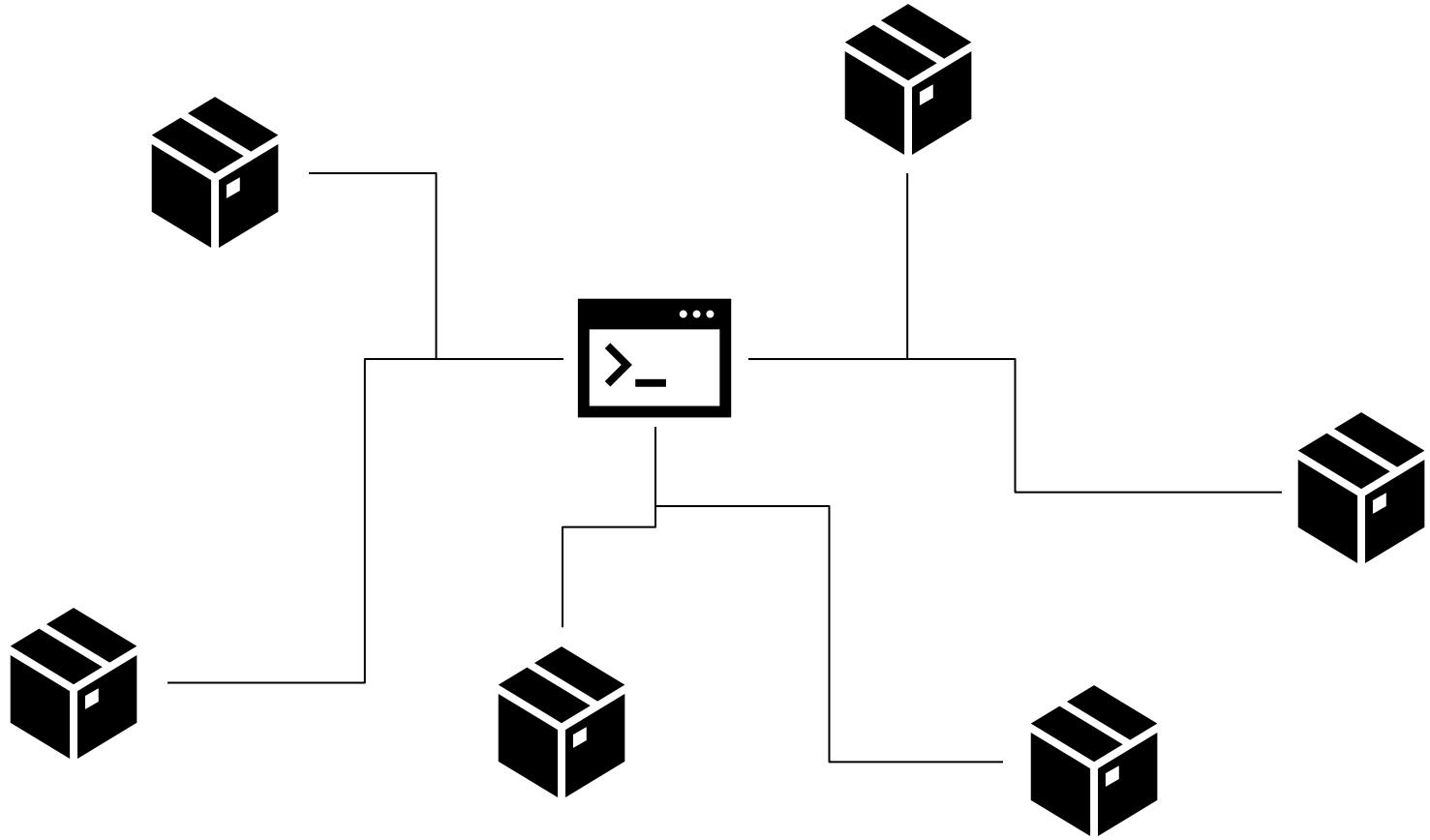
```
// Redact at your logger (example: Serilog)
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .Destructure.ByMaskingProperties("password",
        "authorization",
        "cookie",
        "set-cookie")
    .CreateLogger();
```

Dependency and Supply-Chain Risks

Common Mistake #8

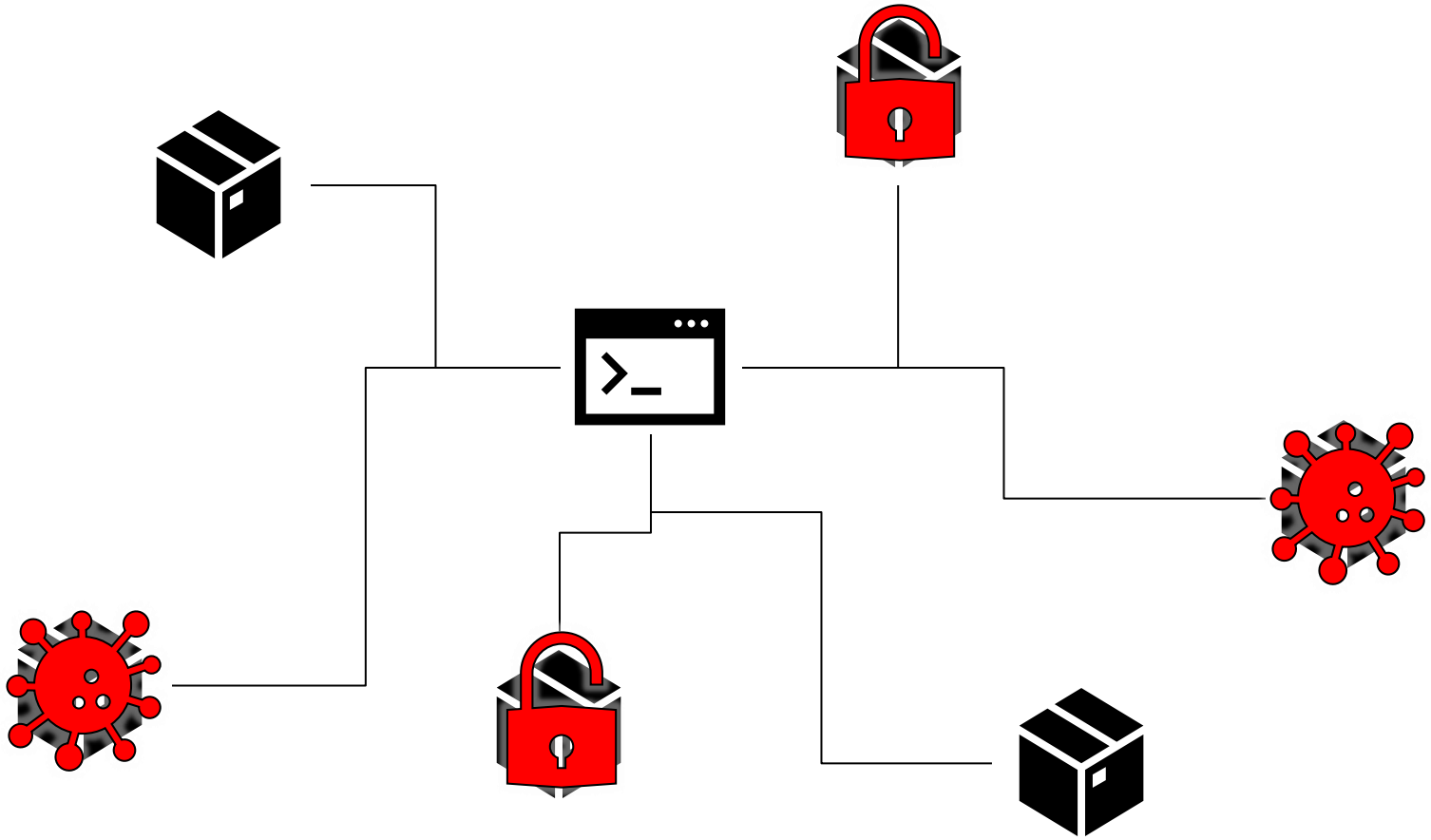
TRAILHEAD
TECHNOLOGY PARTNERS

Security



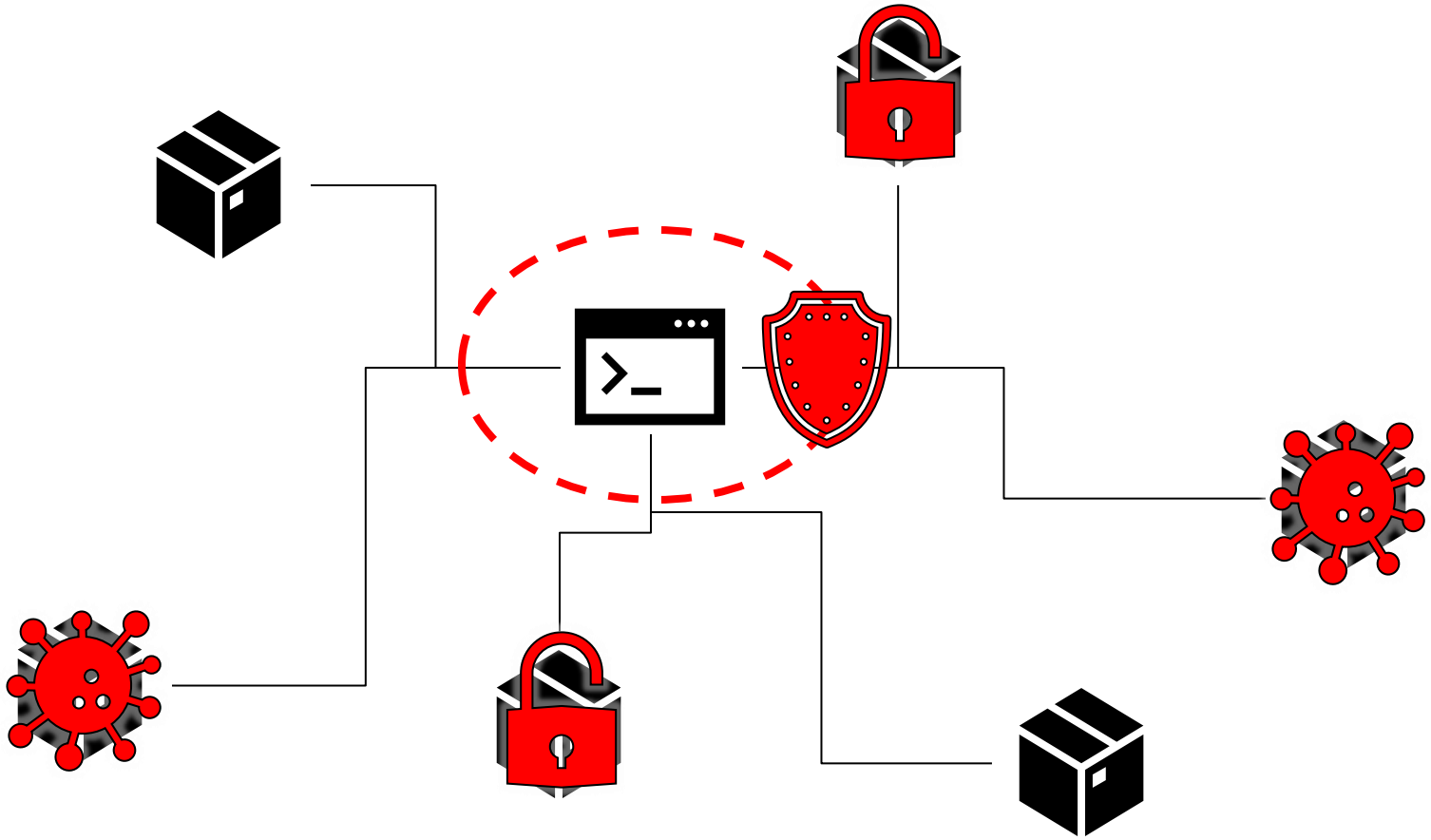
TRAILHEAD
TECHNOLOGY PARTNERS

Security



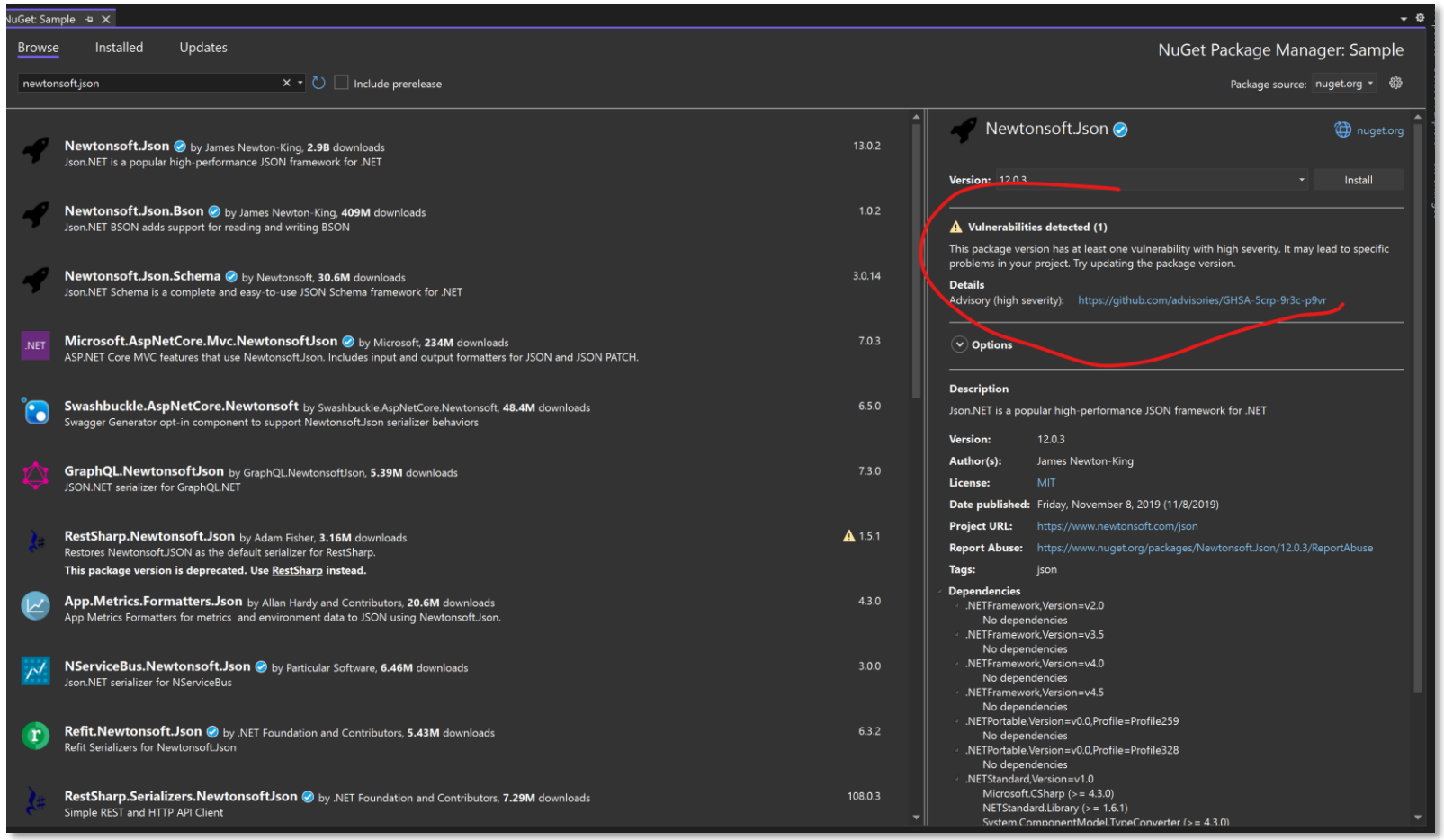
TRAILHEAD
TECHNOLOGY PARTNERS

Security



TRAILHEAD
TECHNOLOGY PARTNERS

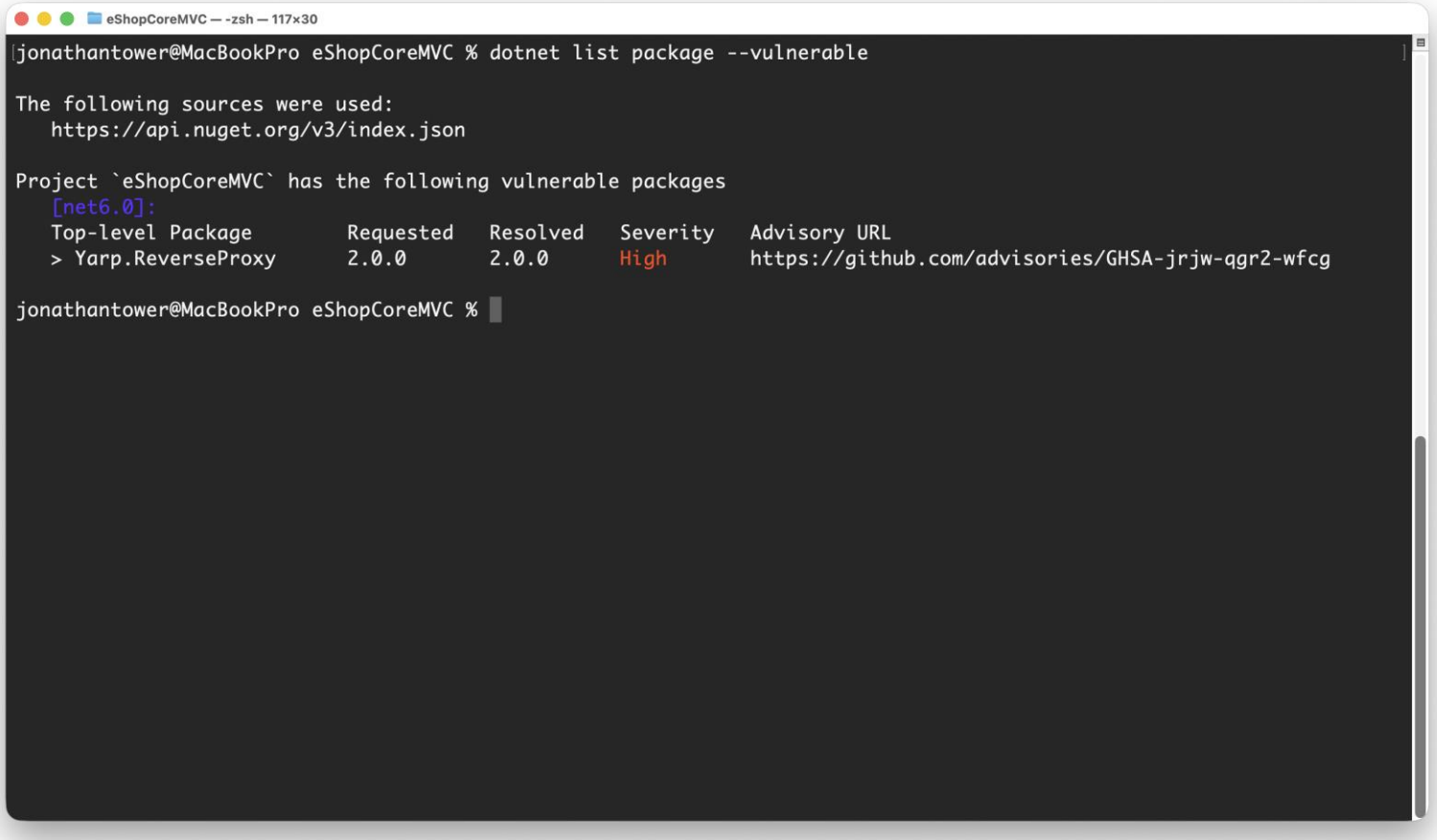
Security



TRAILHEAD
TECHNOLOGY PARTNERS

Security

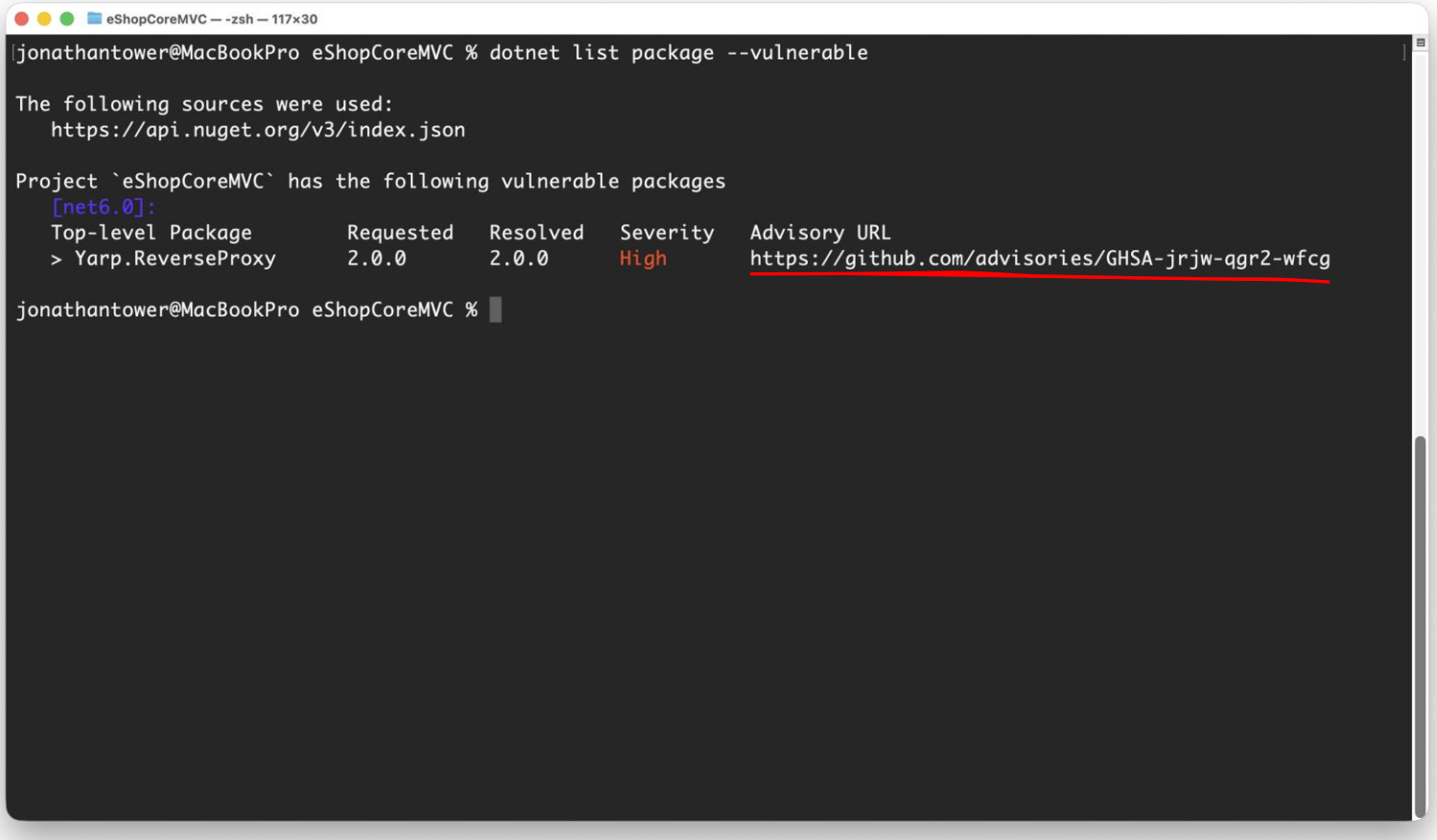
`dotnet list package --vulnerable`



TRAILHEAD TECHNOLOGY PARTNERS

Security

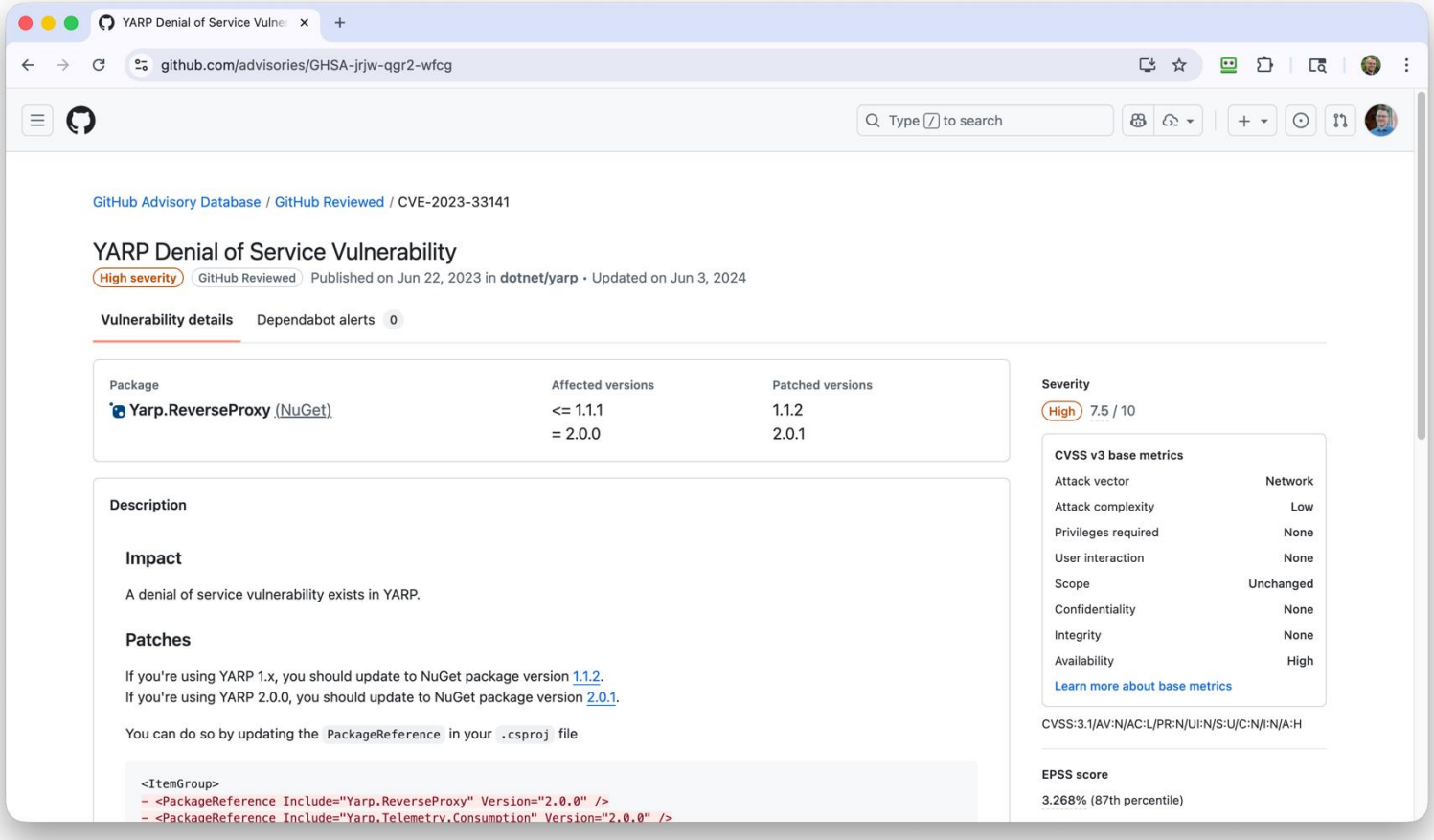
`dotnet list package --vulnerable`



TRAILHEAD TECHNOLOGY PARTNERS

Safety & Reliability: Security

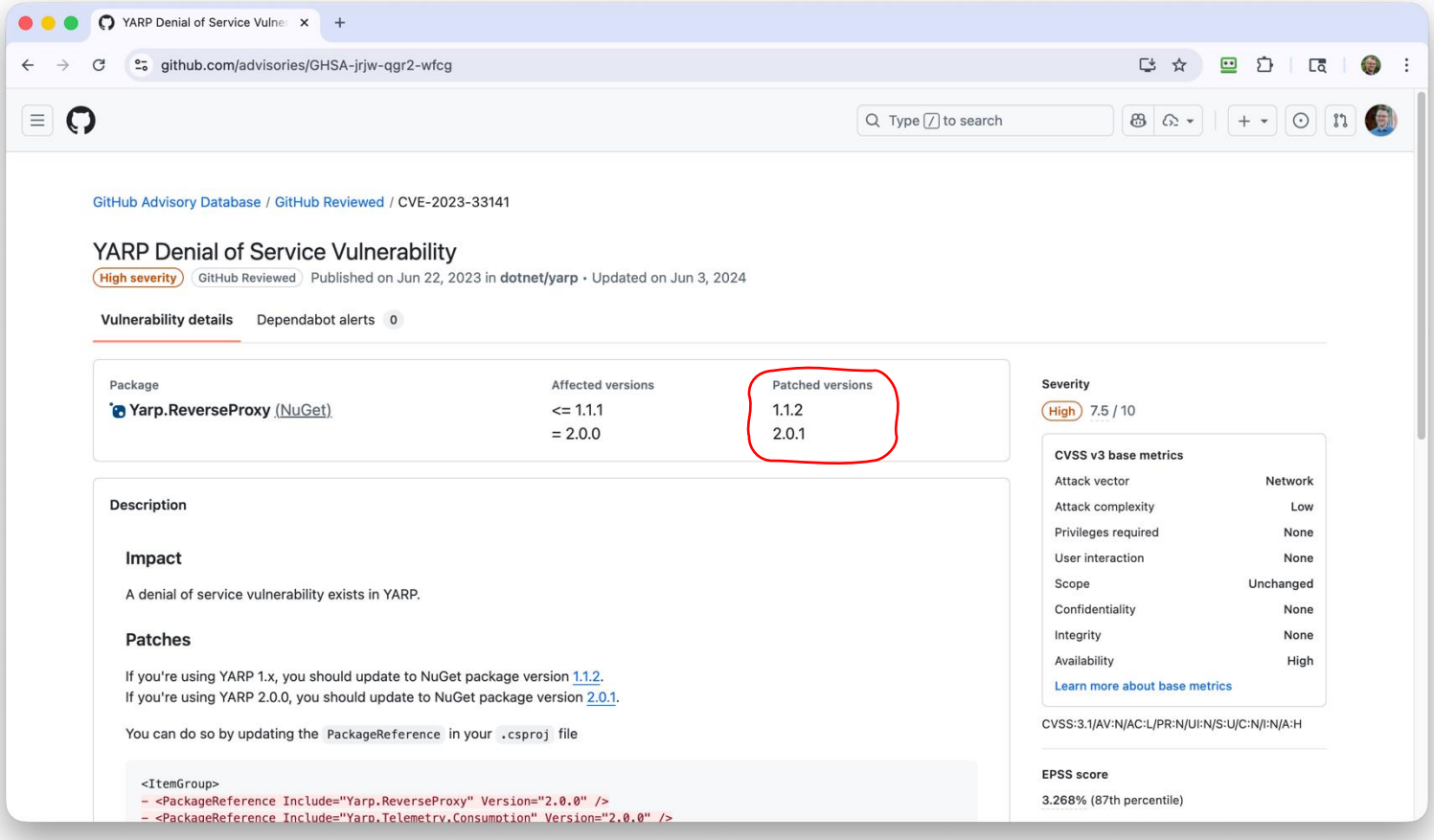
```
dotnet list package --vulnerable
```



TRAILHEAD TECHNOLOGY PARTNERS

Safety & Reliability: Security

```
dotnet list package --vulnerable
```



TRAILHEAD TECHNOLOGY PARTNERS

Security

```
# npm audit report

ansi-html  *
Severity: high
Uncontrolled Resource Consumption in ansi-html - https://github.com/advisories/GHSA-whgm-jr23-g3j9
fix available via `npm audit fix --force`
Will install @vue/cli-plugin-babel@3.12.1, which is a breaking change
node_modules/ansi-html
  webpack-dev-server  2.0.0-beta - 4.1.0
  Depends on vulnerable versions of ansi-html
  Depends on vulnerable versions of chokidar
  Depends on vulnerable versions of http-proxy-middleware
  Depends on vulnerable versions of yargs
  node_modules/webpack-dev-server
    @vue/cli-service  <=5.0.0-beta.3
    Depends on vulnerable versions of @vue/cli-plugin-router
    Depends on vulnerable versions of @vue/cli-shared-utils
    Depends on vulnerable versions of copy-webpack-plugin
    Depends on vulnerable versions of cssnano
    Depends on vulnerable versions of globby
    Depends on vulnerable versions of webpack-dev-server
    node_modules/@vue/cli-service
      @vue/cli-plugin-babel  3.4.0 - 4.5.13
      Depends on vulnerable versions of @vue/cli-service
      Depends on vulnerable versions of @vue/cli-shared-utils
      node_modules/@vue/cli-plugin-babel
      @vue/cli-plugin-router  <=4.5.13
      Depends on vulnerable versions of @vue/cli-service
      node_modules/@vue/cli-plugin-router
      @vue/cli-plugin-typescript  <=5.0.0-alpha.1
      Depends on vulnerable versions of @vue/cli-service
      Depends on vulnerable versions of @vue/cli-shared-utils
      Depends on vulnerable versions of fork-ts-checker-webpack-plugin
      Depends on vulnerable versions of globby
      node_modules/@vue/cli-plugin-typescript
      @vue/cli-plugin-vuex  <=4.5.13
      Depends on vulnerable versions of @vue/cli-service
      node_modules/@vue/cli-plugin-vuex
```

`npm audit`

Security

OWASP Dependency-Track

Codebase

SBOM Tool

SBOMs
(CycloneDX format)

OWASP Dependency-Track

TRAILHEAD
TECHNOLOGY PARTNERS

Transport Layer and Secrets Management

Common Mistake #9

HTTPS and TLS posture 

```
builder.WebHost.ConfigureKestrel(o => o.ListenAnyIP(80)); // HTTP only
app.UseHttpsRedirection();
```

HTTPS and TLS posture

```
builder.WebHost.ConfigureKestrel(o => o.ListenAnyIP(443, l => l.UseHttps()));
app.UseHsts();
```

Secrets In Config 

```
// appsettings.json
{
    "Jwt": {
        "SigningKey": "secret123" // ❌
    },
    "Db": {
        "Password": "P@ss!" // ❌
     }
}
```

Secrets In Config 

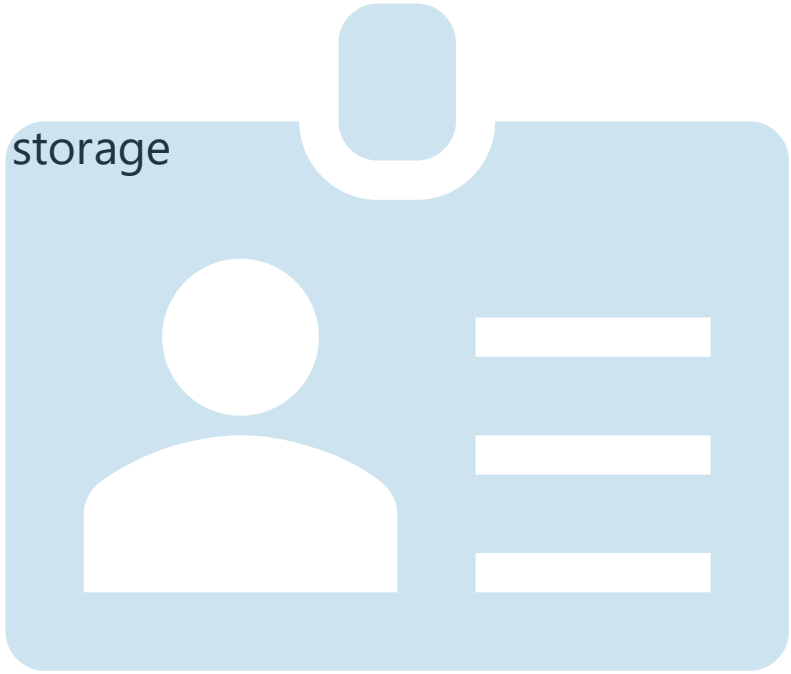
```
// Dev: User Secrets
dotnet user-secrets set "Db:Password" "..." // not committed to repo

// Cloud: Azure Key Vault + Managed Identity
using Azure.Identity;
builder.Configuration
    .AddAzureKeyVault(new Uri("https://<vault>.vault.azure.net/"),
                      new DefaultAzureCredential()); // ✅ no static keys
```

Developer-Friendly Best Practices Checklist

TRAILHEAD
TECHNOLOGY PARTNERS

Authentication

- ✅ Use JWT
- ✅ Use proven implementations for auth, tokens, and password storage
- ✅ Use max retries & lockout
- ✅ Encrypt sensitive data at rest



Inspiration: https://github.com/shieldfy/API-Security-Checklist

JSON Web Token

- ✅ Use a strong, random signing secret and rotate it
- ✅ Enforce the signing algorithm server-side
- ✅ Use short token lifetimes, refresh tokens
- ✅ Never store sensitive data in JWT payloads.
- ✅ Keep claims minimal

Inspiration: https://github.com/shieldfy/API-Security-Checklist

Access

- [x] Rate-limit to prevent brute force attacks
- [x] Enforce HTTPS (TLS 1.2+) with strong ciphers; verify Host/SNI
- [x] Enable HSTS to prevent SSL stripping
- [x] Disable directory indexing/listing
- [x] Disable OpenAPI for private APIs
- [x] Restrict private APIs to safelisted IPs/hosts

Inspiration: https://github.com/shieldfy/API-Security-Checklist

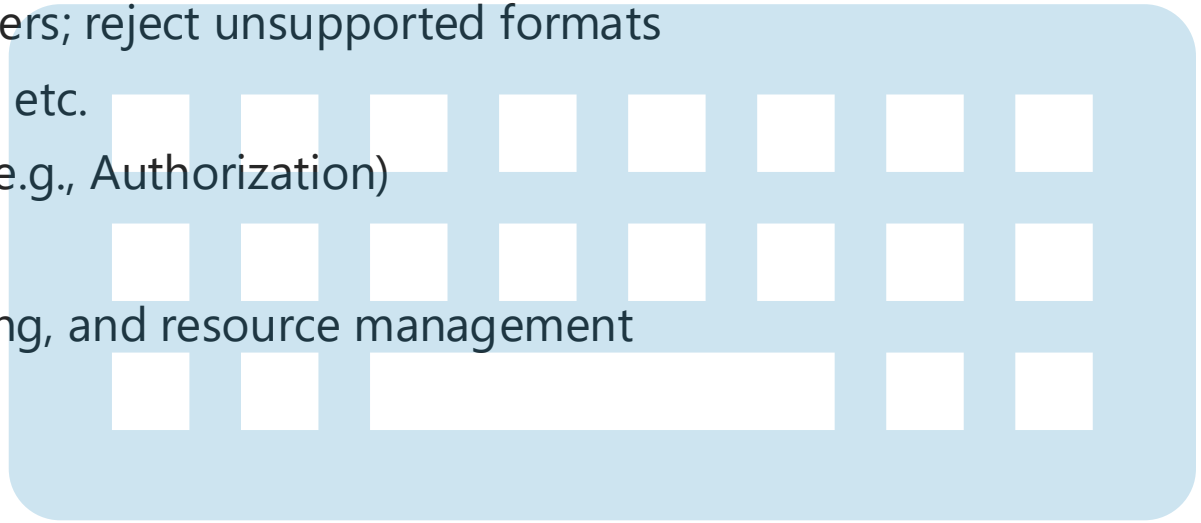
OAuth

- ✅ Validate redirect_uri server-side against a safelist
- ✅ Exchange authorization codes, not tokens (response_type=code)
- ✅ Use a random state parameter to prevent CSRF
- ✅ Define a default scope and validate requested scopes per application

Inspiration: https://github.com/shieldfy/API-Security-Checklist

Input

- ✅ Use correct HTTP methods (GET, POST, PUT/PATCH, DELETE); reject invalid ones
- ✅ Validate Accept and request Content-Type headers; reject unsupported formats
- ✅ Validate all user input to prevent XSS, SQLi, RCE, etc.
- ✅ Never pass sensitive data in URLs; use headers (e.g., Authorization)
- ✅ Encrypt data only on the server side
- ✅ Consider an API Gateway for caching, rate limiting, and resource management
  - Amazon API Gateway
  - Azure API Management
  - Cloudflare



Inspiration: https://github.com/shieldfy/API-Security-Checklist

Processing

- ✅ Protect all endpoints with authentication
- ✅ Use /me/... instead of exposing raw user IDs
- ✅ Use UUIDs instead of auto-increment IDs
- ✅ Disable XML entity parsing (prevent XXE)
- ✅ Disable entity expansion in XML/YAML (prevent “Billion Laughs” attacks)
- ✅ Use a CDN for file uploads
- ✅ Offload heavy processing to workers/queues for faster responses
- ✅ Turn off debug mode in production
- ✅ Enable non-executable stacks when supported

Inspiration: https://github.com/shieldfy/API-Security-Checklist

Output

- ✅ Send security headers (X-Content-Type-Options, X-Frame-Options, CSP)
- ✅ Remove fingerprinting headers (X-Powered-By, Server, etc.)
- ✅ Set the correct Content-Type in responses
- ✅ Never return sensitive data in responses
- ✅ Use proper HTTP status codes for each operation

Inspiration: https://github.com/shieldfy/API-Security-Checklist

CI & CD

- ✅ Cover design and code with unit/integration tests
- ✅ Require peer code reviews (no self-approval)
- ✅ Scan all components and dependencies before release
- ✅ Continuously run static/dynamic security tests
- ✅ Monitor dependencies/OS for known vulnerabilities
- ✅ Plan rollback strategies for deployments

Inspiration: https://github.com/shieldfy/API-Security-Checklist

Monitoring

- ✅ Centralize logins across all services
- ✅ Monitor traffic, errors, and requests with agents
- ✅ Set up alerts (SMS, Slack, Email, etc.)
- ✅ Avoid logging sensitive data (passwords, cards, PINs)
- ✅ Use IDS/IPS to monitor API requests and instances

Inspiration: https://github.com/shieldfy/API-Security-Checklist

The P.A.S.S. Framework

- P = Purpose-built
- A = Authenticate & Authorize
- S = Shape your data (API = UI)
- S = Secure defaults

Before You Ship...

- Ask yourself: does my API P.A.S.S. the test?

Summing Up

1. todo

TRAILHEAD
TECHNOLOGY PARTNERS

Thanks! Questions?

Jonathan "J." Tower

- Microsoft MVP in .NET
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- jtowermi
- Jonathan “J.” Tower

EXPERT
Consultation

bit.ly/th-offer

github.com/trailheadtechnology/api-security